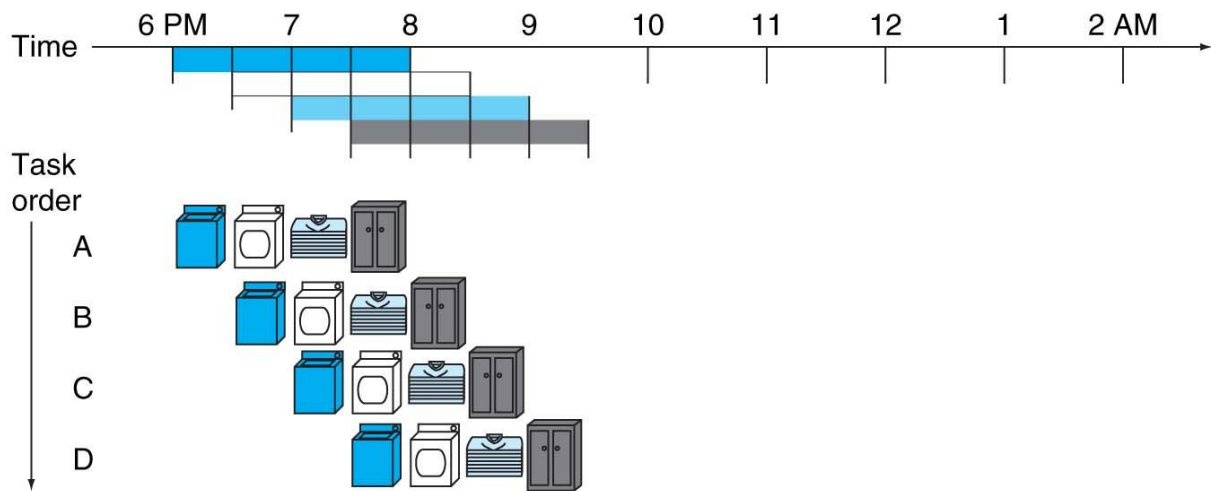
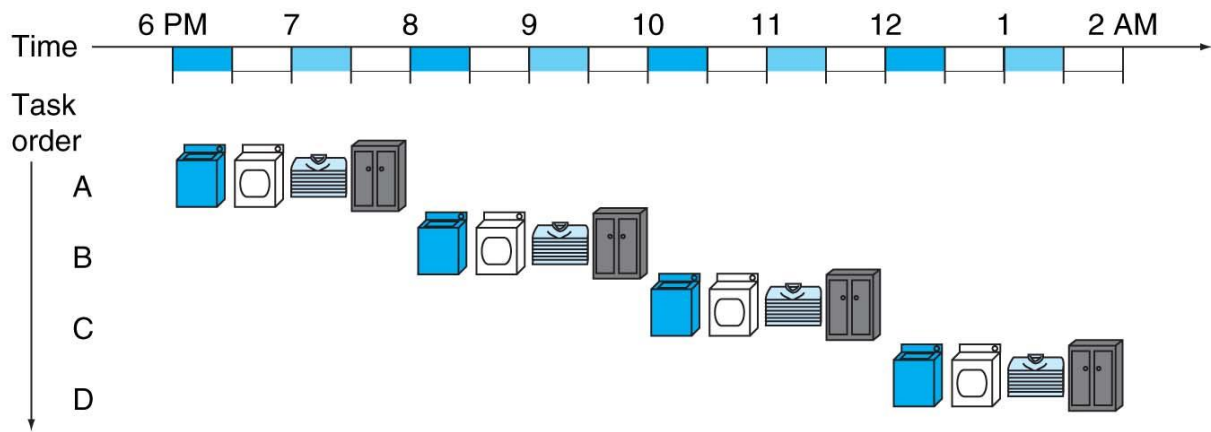


Chapter 2

Instruction-Level Parallelism (ILP)





Data Dependences

- FLD F0, 0(R1)
- FADD.D F4, F0, F2



Name Dependences

FDIV.D F2, **F6**, F4

FADD.D **F6**, F0, F12

FSUB.D F8, **F6**, F14

DIV&ADD: Anti-dependence

Change **F6** as **S**:

FDIV.D F2, **F6**, F4

FADD.D **S**, F0, F12

FSUB.D F8, **S**, F14

FDIV.D **F2**, F6, F4

FADD.D F6, F0, F12

FSUB.D **F2**, F6, F14

DIV&SUB: Output-dependence

Change **F2** as **S**:

FDIV.D **F2**, F0, F4

FADD.D F6, F0, F12

FSUB.D **S**, F6, F14



Control Dependences

```
if p1 {  
    Statement 1  
}
```

Statement

```
if p2 {  
    Statement 2  
}
```



Hazards

- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
 - A required resource is busy
- Data hazard
 - Need to wait for previous instruction to complete its data read/write
- Control hazard
 - Deciding on control action depends on previous instruction

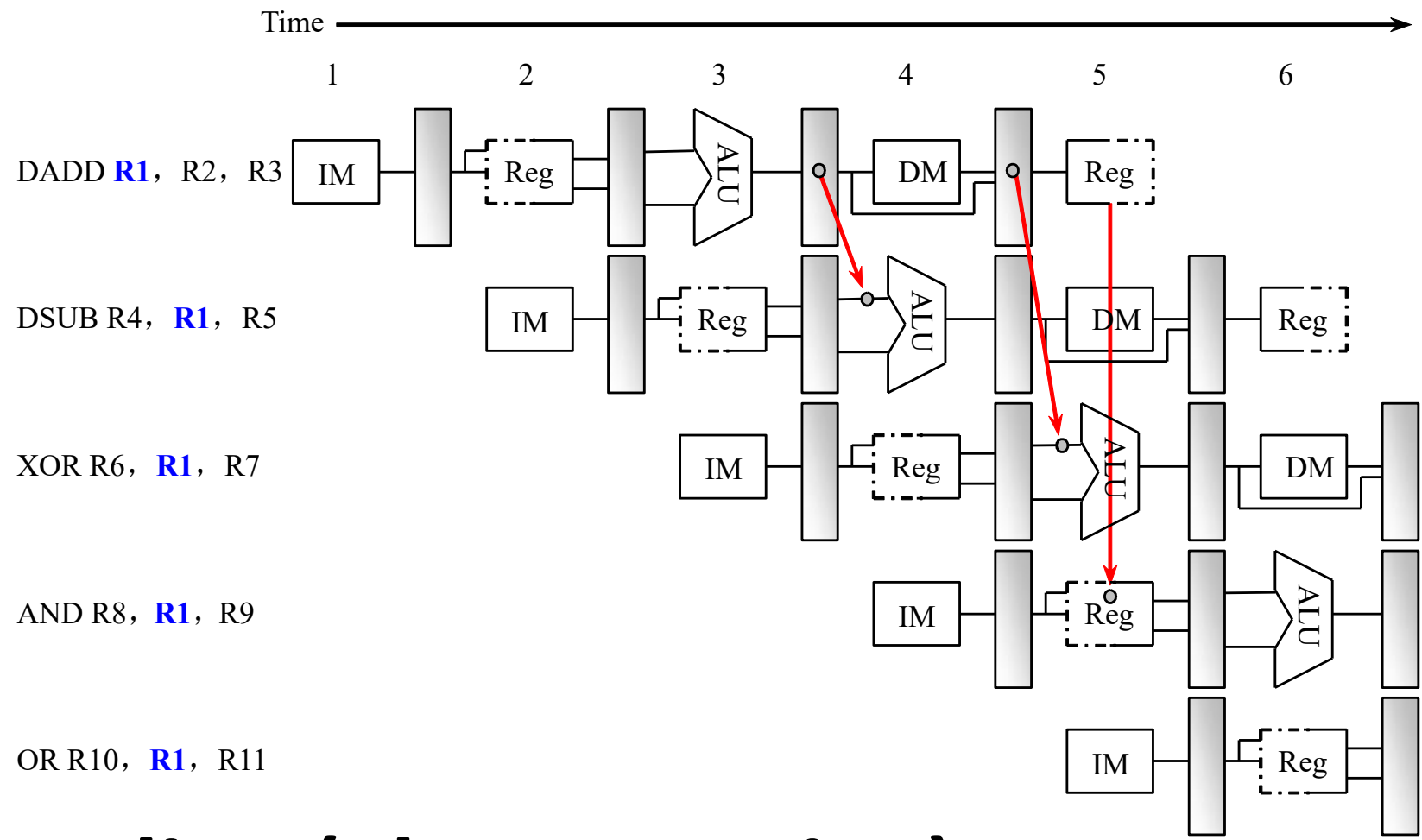


Data Hazards

• Read after write: RAW	FADD.D	F6, F0, F12
	FSUB.D	F8, F6, F14
• Write after read: WAR	FDIV.D	F2, F6, F4
	FADD.D	F6, F0, F12
• Write after write: WAW	FDIV.D	F2, F0, F4
	FSUB.D	F2, F6, F14



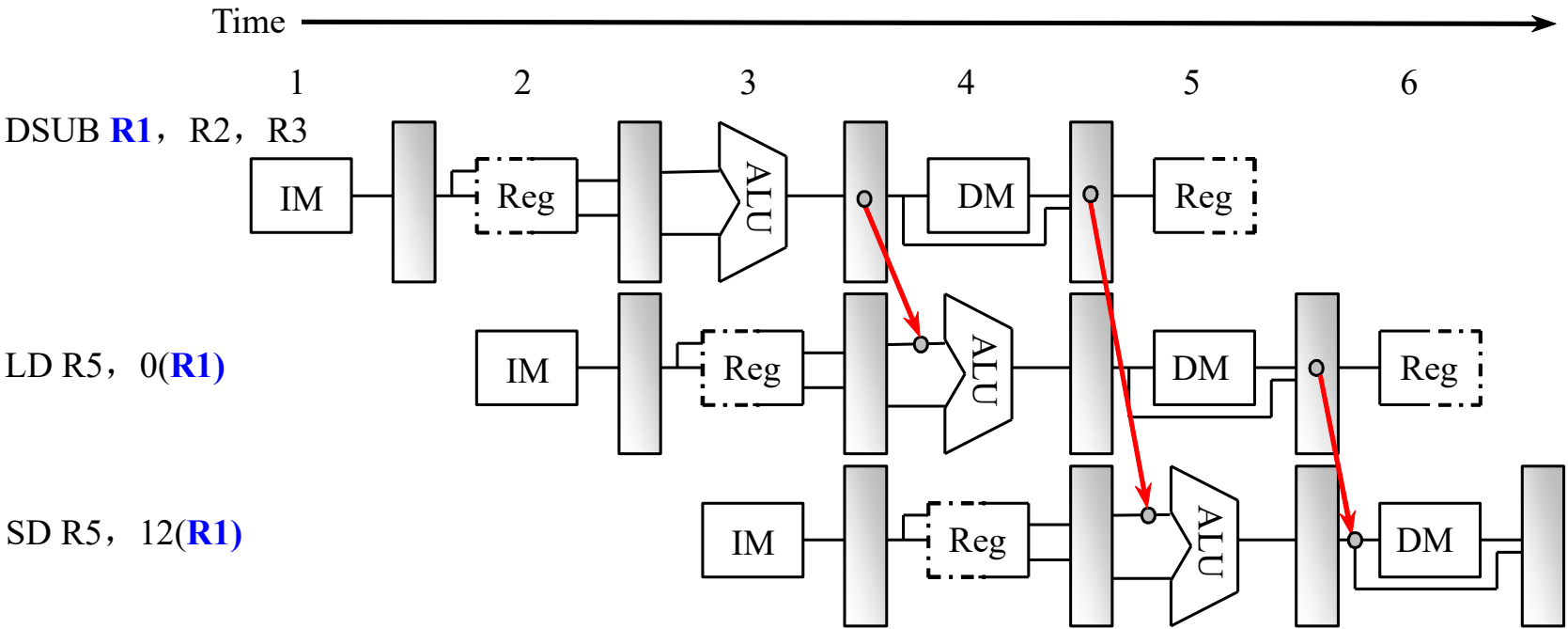
Data Hazards



Forwarding (aka Bypassing)



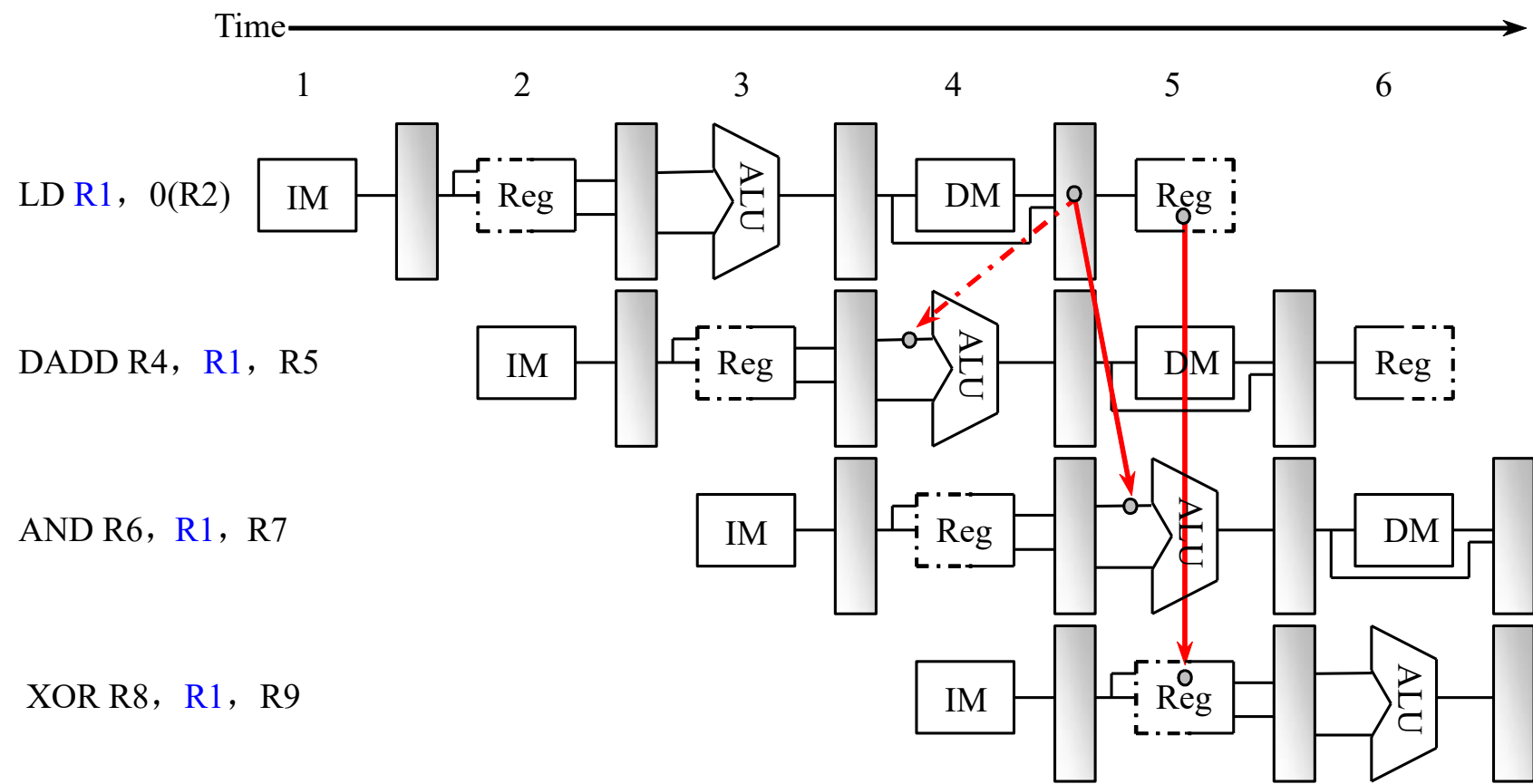
Data Hazards



Forwarding (aka Bypassing)



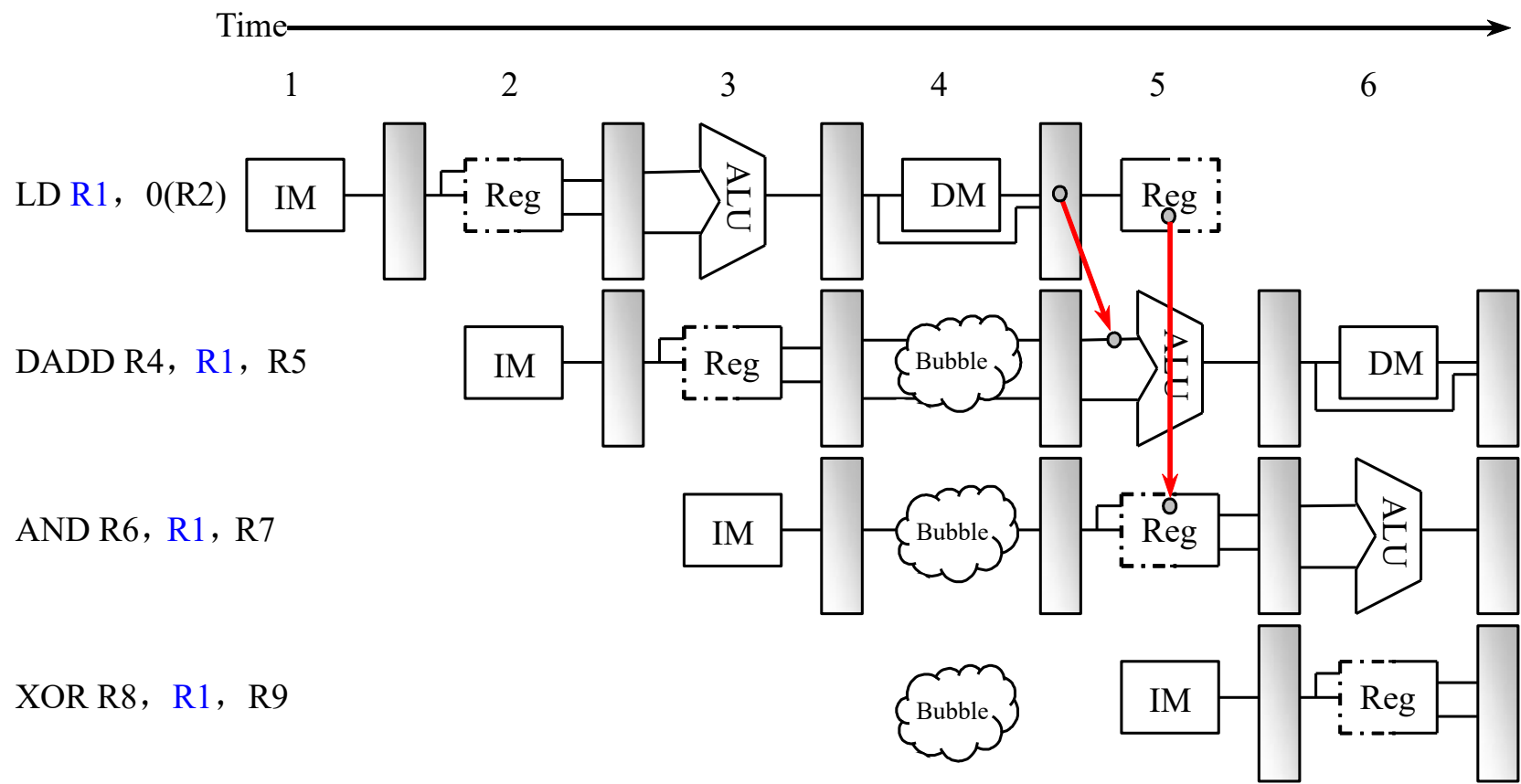
Data Hazards



Forwarding (aka Bypassing)



Data Hazards



Forwarding with bubble



Data Hazards

LD R1, 0 (R2)	IF	ID	EX	MEM	WB			
DADD R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
XOR R8, R1, R9				IF	ID	EX	MEM	WB

LD R1, 0 (R2)	IF	ID	EX	MEM	WB			
DADD R4, R1, R5		IF	ID	stall	EX	MEM	WB	
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB
XOR R8, R1, R9				stall	IF	ID	EX	MEM

Forwarding with bubble



Data Hazards

A = B + C

LD Rb, B	IF	ID	EX	MEM	WB				
LD Rc, C		IF	ID	EX	MEM	WB			
DADD Ra, Rb, Rc			IF	ID	stall	EX	MEM	WB	
SD Ra , A				IF	stall	ID	EX	MEM	WB

Code Scheduling to Avoid Stalls



Data Hazards

$A = B + C$
 $D = E - F$

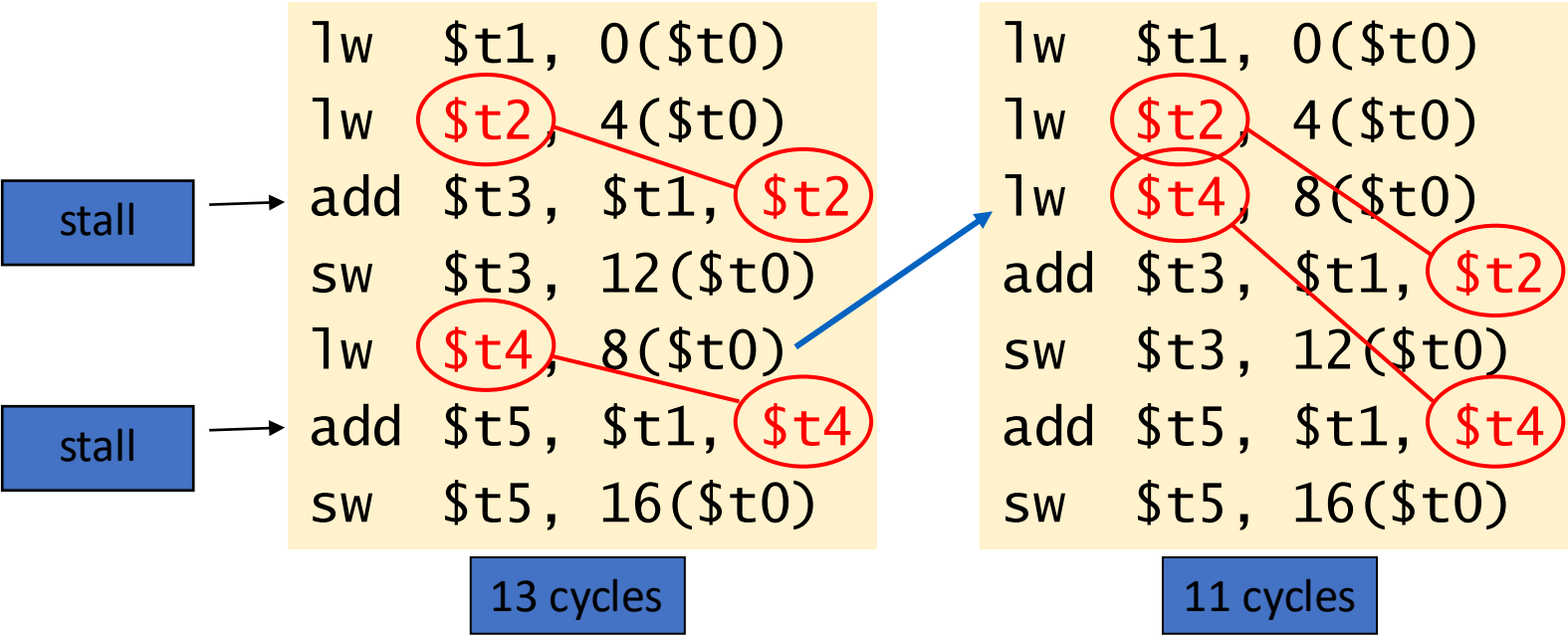
Before Scheduling	After Scheduling
LD Rb, B	
LD Rc, C	
<u>DADD Ra, Rb, Rc</u>	
SD Ra, A	
LD Re, E	
LD Rf, F	
<u>DSUB Rd, Re, Rf</u>	
SD Rd, D	

Code Scheduling to Avoid Stalls



Data Hazards

- Reorder code to avoid use of load result in the next instruction
- C code for $A = B + E$; $C = B + F$;



Code Scheduling to Avoid Stalls



Control Hazards

- Branch determines flow of control
 - Fetching next instruction depends on branch outcome
 - Pipelining can't always fetch correct instruction
 - Still working on ID stage of branch
- In RISC-V pipelining
 - Need to compare registers and compute target early in the pipelining
 - Add hardware to do it in ID stage

Unconditional Jump

Conditional Branch

Jal - Jump and Link

Jalr - Jump and Link-Register



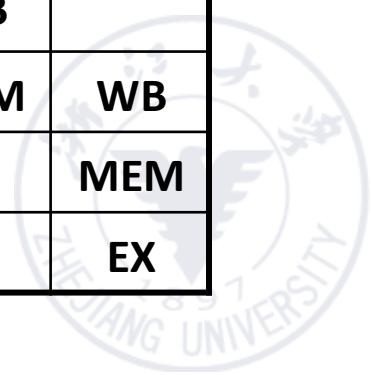
Stall on Branch

Branch taken

Branch	IF	ID	EX	MEM	WB					
Target		IF	stall	stall	IF	ID	EX	MEM	WB	
Target+1						IF	ID	EX	MEM	WB
Target+2							IF	ID	EX	MEM
Target+3								IF	ID	EX

Branch not taken

Branch	IF	ID	EX	MEM	WB					
Instruction i		IF	stall	stall	IF	ID	EX	MEM	WB	
Instruction i+1						IF	ID	EX	MEM	WB
Instruction i+2							IF	ID	EX	MEM
Instruction i+3								IF	ID	EX



Stall on Branch

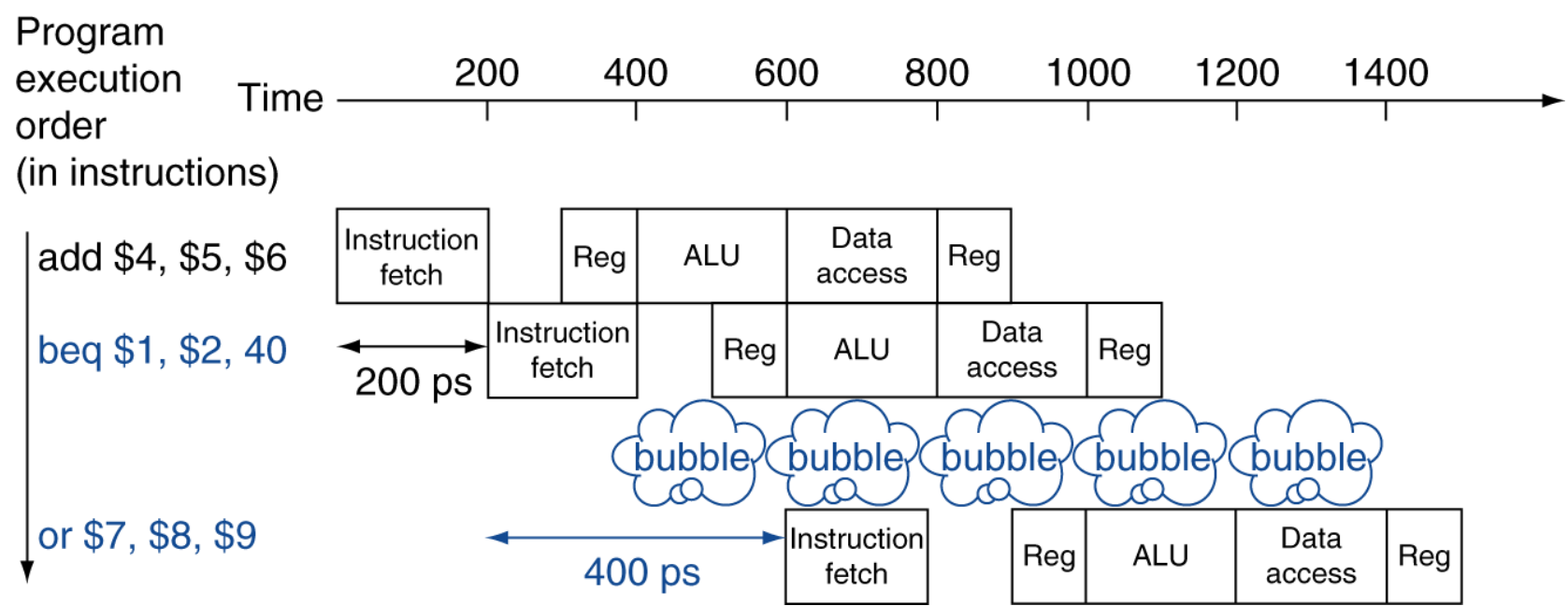
Branch causes a stall

Branch	IF	ID	EX	MEM	WB					
Instruction i		IF	IF	ID	EX	MEM	WB			
Instruction i+1				IF	ID	EX	MEM	WB		
Instruction i+2					IF	ID	EX	MEM	WB	
Instruction i+3						IF	ID	EX	MEM	WB



Control Hazards

- Wait until branch outcome determined before fetching next instruction



Branch Prediction

- Longer pipelines can't readily determine branch outcome early
 - Stall penalty becomes unacceptable
- Predict outcome of branch
 - Only stall if prediction is wrong
- In RISC-V pipeline
 - Can predict branches not taken
 - Fetch instruction after branch, with no delay



Control Hazards:

More-Realistic Branch Prediction

- Static branch prediction
 - Based on typical branch behavior
 - Example: loop and if-statement branches
 - Predict backward branches taken
 - Predict forward branches not taken
- Dynamic branch prediction
 - Hardware measures actual branch behavior
 - e.g., record recent history of each branch
 - Assume future behavior will continue the trend
 - When wrong, stall while re-fetching, and update history



Reducing Branch Delay

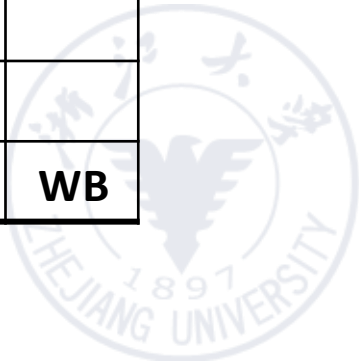
- Predict branch taken
- Predict branch not taken
- Delayed Branch



Predict not taken

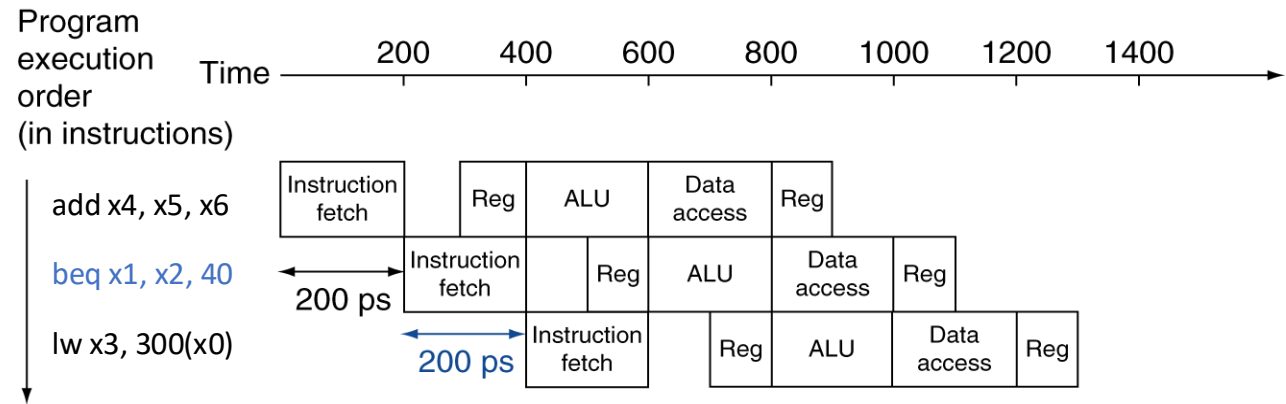
Branch i (taken)	IF	ID	EX	MEM	WB				
Instruction i+1		IF	stall	stall	stall	stall			
Branch target j			IF	ID	EX	MEM	WB		
Branch target j+1				IF	ID	EX	MEM	WB	
Branch target j+2					IF	ID	EX	MEM	WB

Branch i (not taken)	IF	ID	EX	MEM	WB				
Instruction i+1		IF	ID	EX	MEM	WB			
Instruction i+2			IF	ID	EX	MEM	WB		
Instruction i+3				IF	ID	EX	MEM	WB	
Instruction i+4					IF	ID	EX	MEM	WB

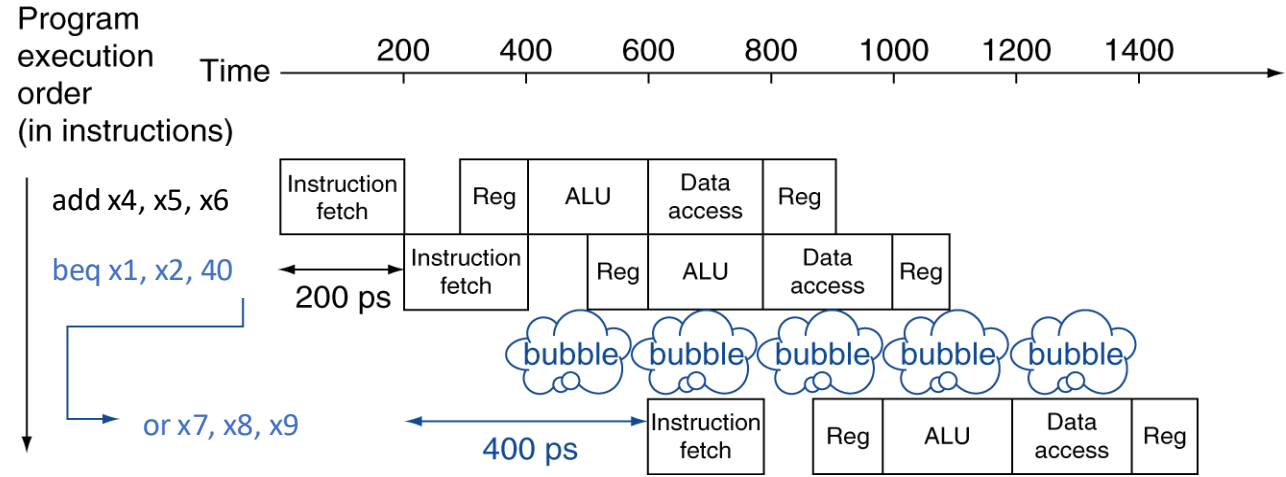


RISC-V with Predict Not Taken

Prediction correct



Prediction incorrect



How to Reduce Branch Delay

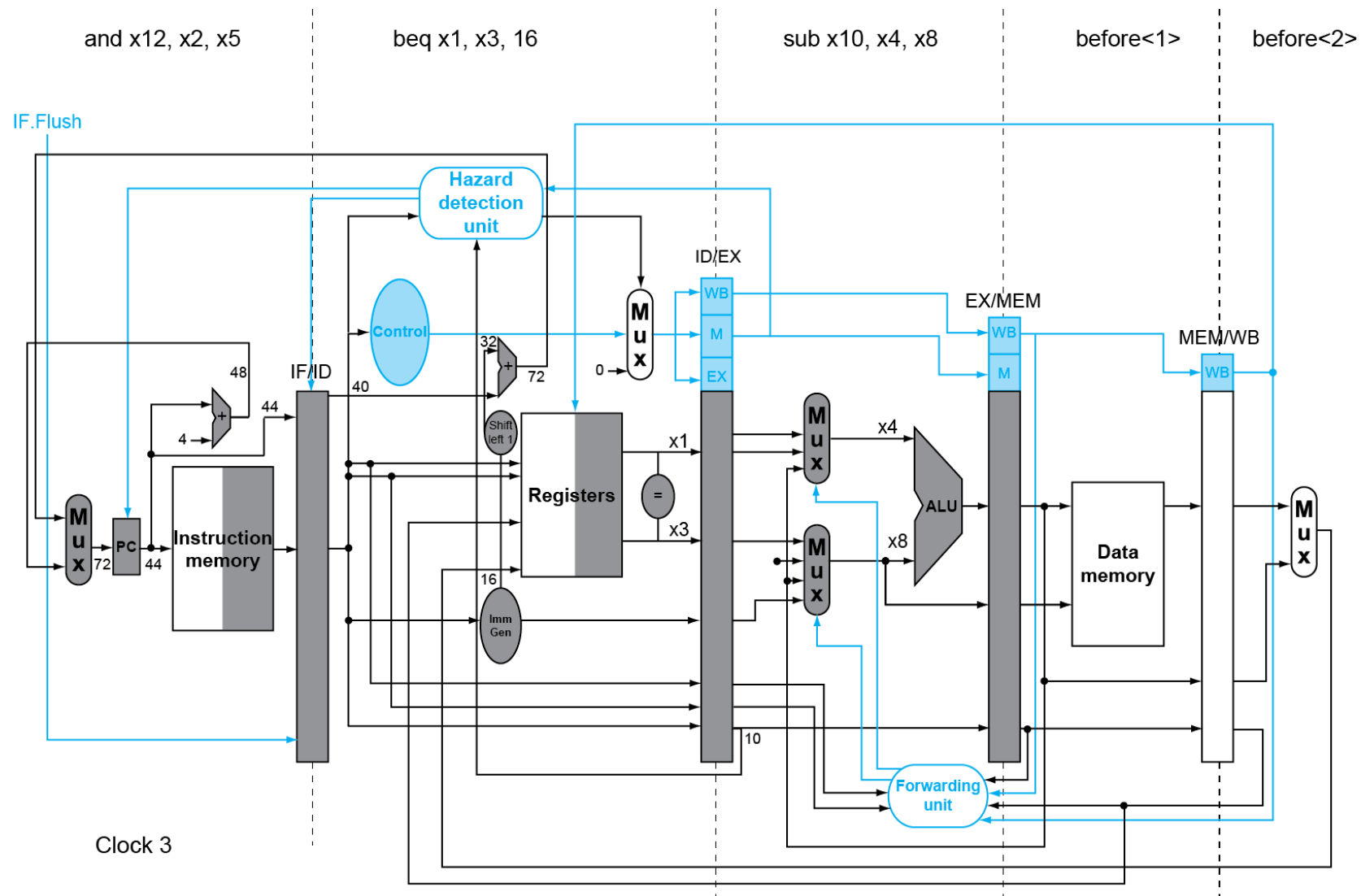
- Example: branch taken

```

36:  sub  x10, x4, x8
40:  beq  x1,  x3, 16    // PC-relative branch
                          // to 40+16*2=72
44:  and  x12, x2, x5
48:  or   x13, x2, x6
52:  add  x14, x4, x2
56:  sub  x15, x6, x7

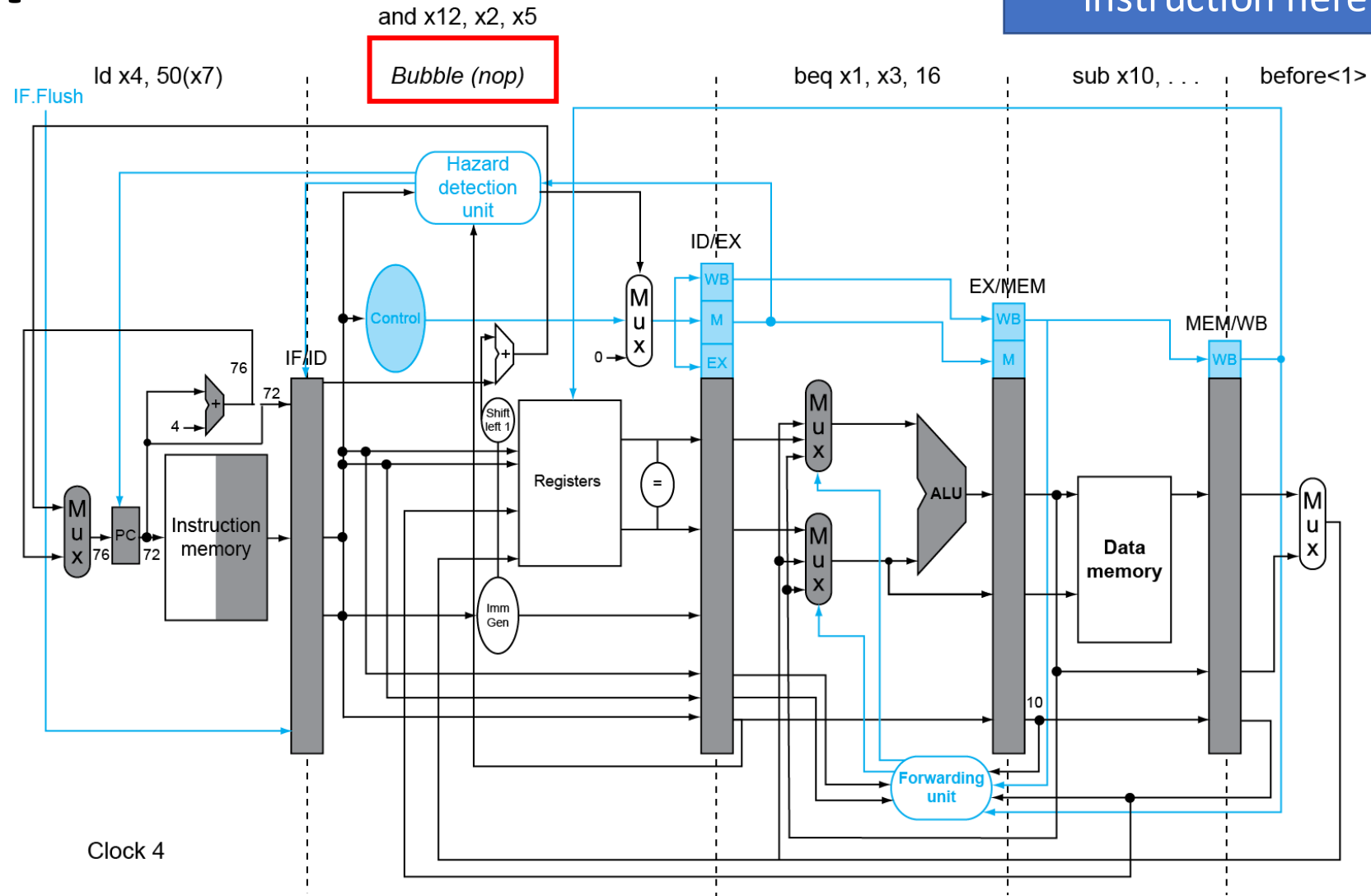
72:  ...
    ld   x4, 50(x7)
    
```

Example: Branch Taken



Example: Branch Taken

What is the original instruction here?



Data Hazards for Branches

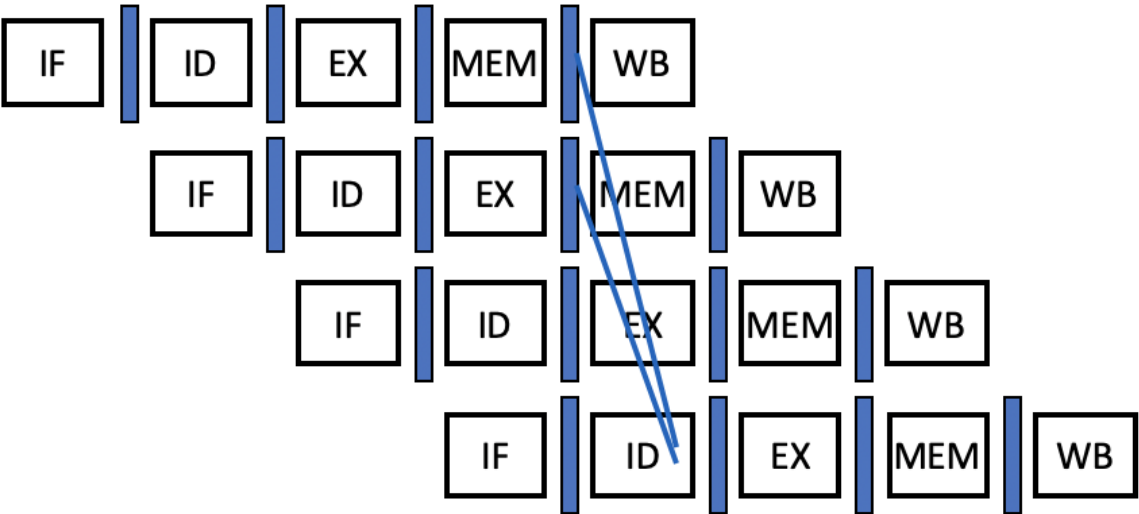
- If a comparison register is a destination of 2nd or 3rd preceding ALU instruction

add \$1, \$2, \$3

add \$4, \$5, \$6

...

beq \$1, \$4, target

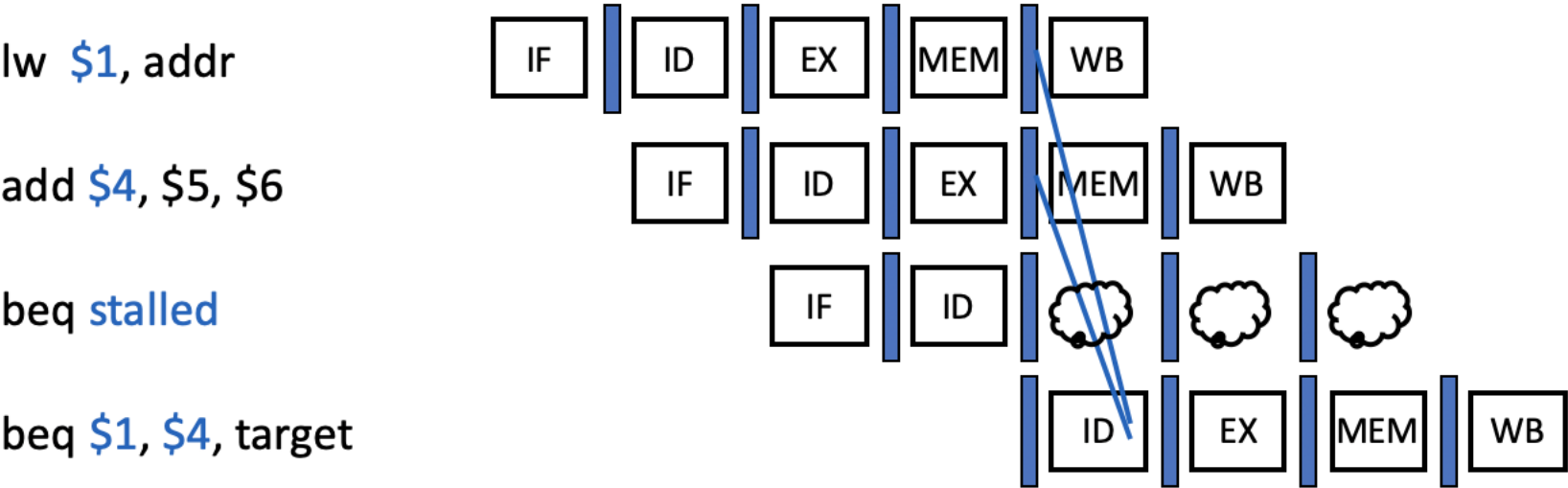


- forwarding



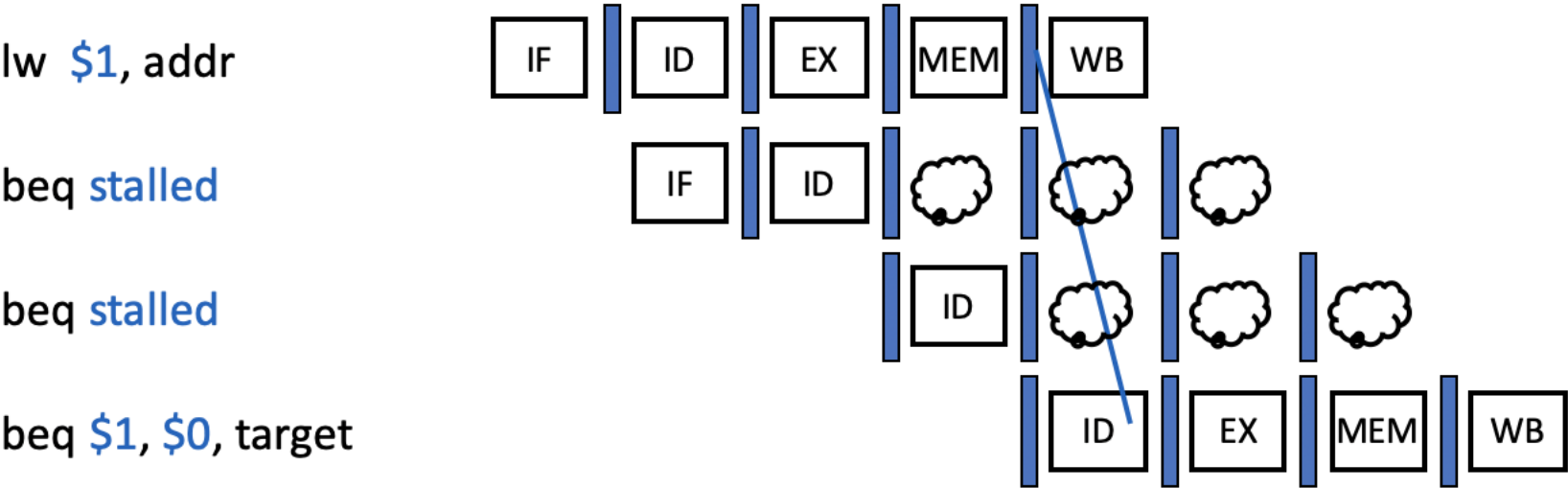
Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
 - Need 1 stall cycle



Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction
 - Need 2 stall cycles



Delay slot

Branch Not taken	Branch i	IF	ID	EX	MEM	WB				
	(Delay slot) instruction i+1		IF	ID	EX	MEM	WB			
	instruction i+2			IF	ID	EX	MEM	WB		
	instruction i+3				IF	ID	EX	MEM	WB	
	instruction i+4					IF	ID	EX	MEM	WB

Branch Taken	Branch i	IF	ID	EX	MEM	WB				
	(Delay slot) instruction i+1		IF	ID	EX	MEM	WB			
	Branch target j			IF	ID	EX	MEM	WB		
	Branch target j+1				IF	ID	EX	MEM	WB	
	Branch target j+2					IF	ID	EX	MEM	WB

Question: Is delay slot a really good design?

- “A **RISC-V ISA** is defined as a base integer ISA, which must be present in any implementation, plus optional extensions to the base ISA.
- The base integer ISAs are very similar to that of the early RISC processors except **with no branch delay slots** and with support for optional variable-length instruction encodings.”

——The RISC-V Instruction Set Manual Volume I

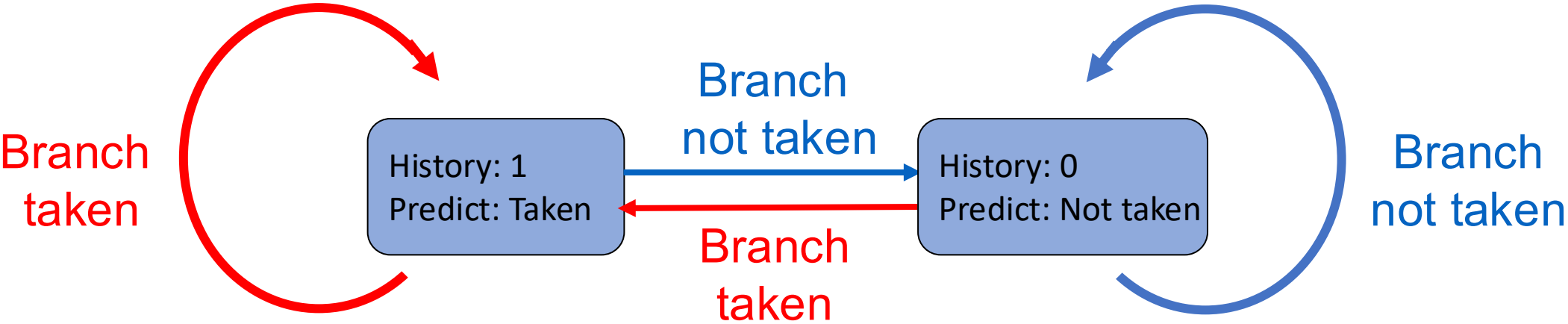


Dynamic Branch Prediction

- In deeper and superscalar pipelines, branch penalty is more significant
- Use dynamic prediction
 - Branch prediction buffer (aka branch history table)
 - Indexed by recent branch instruction addresses
 - Stores outcome (taken/not taken)
 - To execute a branch
 - Check table, expect the same outcome
 - Start fetching from fall-through or target
 - If wrong, flush pipeline and flip prediction

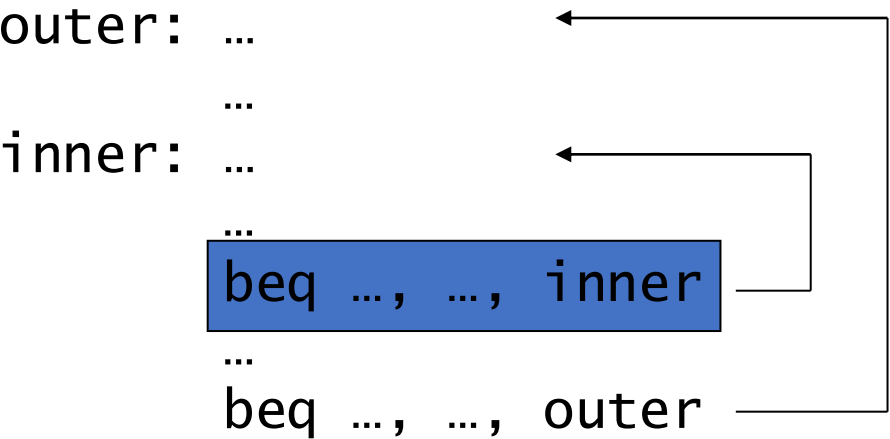


Branch History Table(BHT)



1-Bit Predictor: Shortcoming

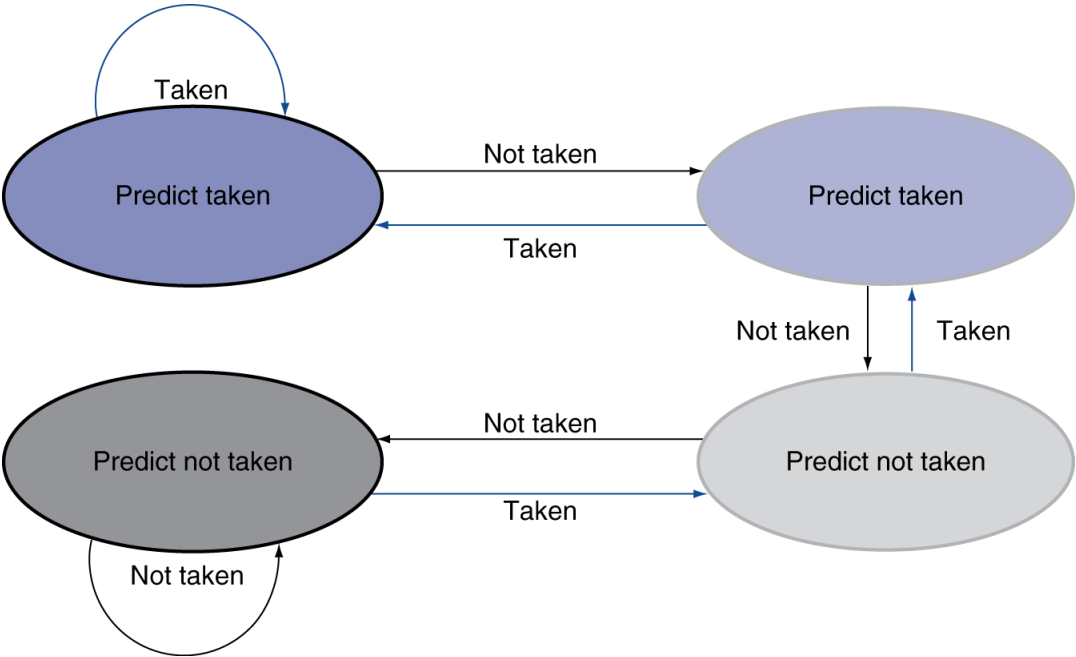
- Inner loop branches mispredicted twice!



- Mispredict as taken on last iteration of inner loop
- Then mispredict as not taken on first iteration of inner loop next time around



2-Bit Predictor



Taken

Not taken

11

10

Taken

Taken

Not taken

Not taken

01

00

Taken

Not taken



2-Bit Predictor: Example

- Inner loop branches mispredicted only once!



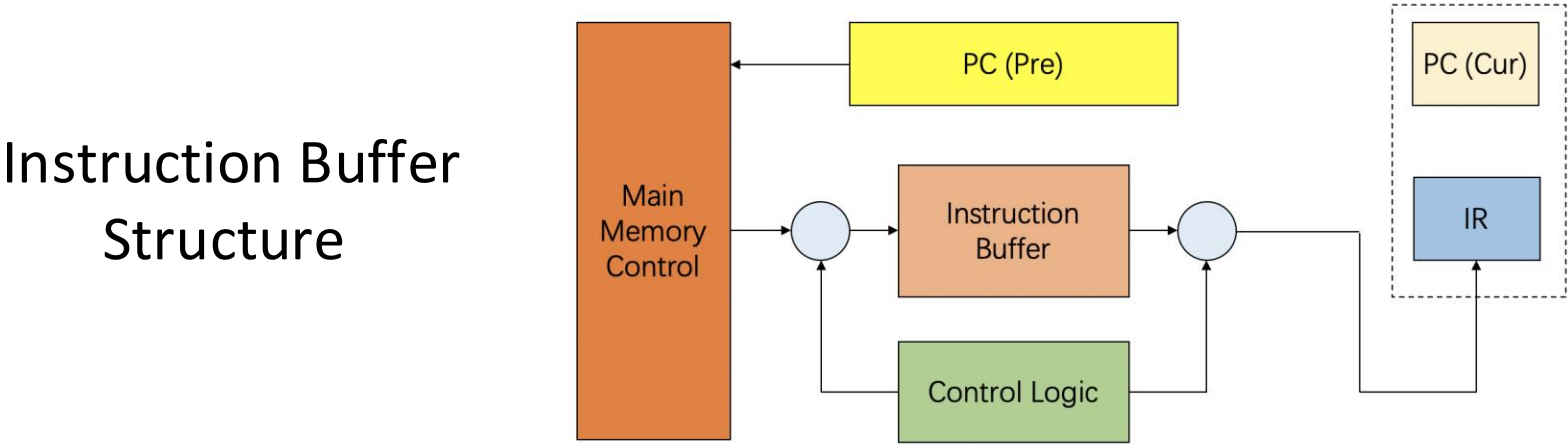
- Only mispredict as taken on last iteration of inner loop

**Buffer plays an important role
in Computer Architecture.**

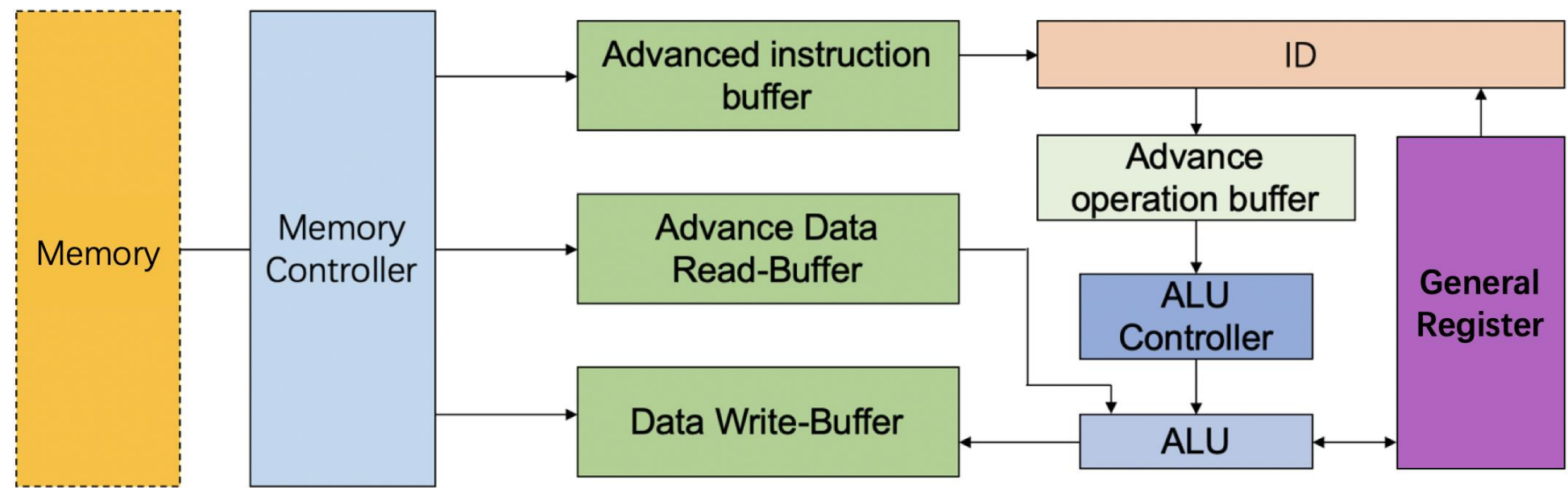


Overlapping execution

- Conflict in access memory
 - Instruction memory & data memory
 - Instruction cache & data cache (same memory): **Harvard structure**
 - Multibody cross structure (same memory with limitations)
 - Adding **instruction buffer** between memory and instruction decode unit



Single Overlapping execution



The structure of processor with advance control

Common features: They work by FIFO, and are composed of a group of several storage units that can be accessed quickly and related control logic.



Advanced Techniques for Instruction Delivery and Speculation

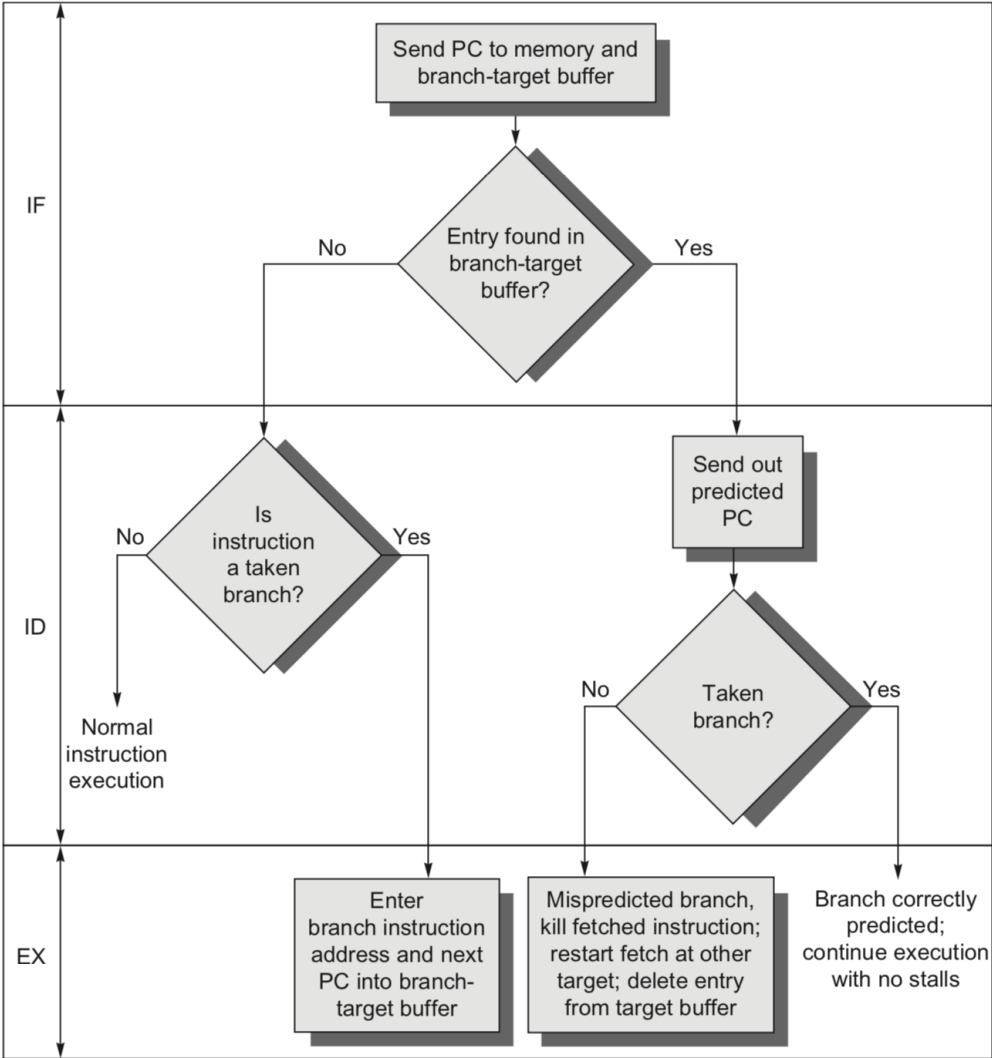
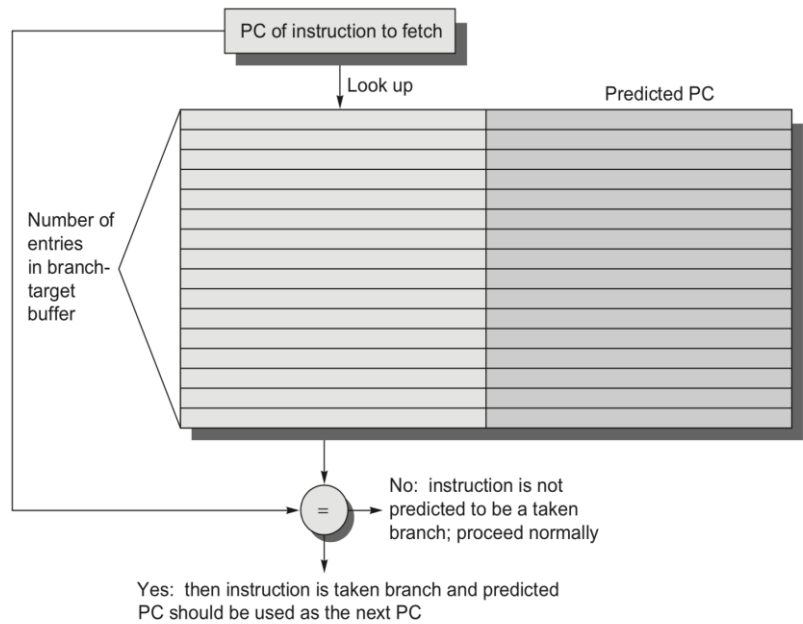
- Increasing Instruction Fetch Bandwidth
 - **Branch-Target Buffers**
- Specialized Branch Predictors: Predicting Procedure Returns, Indirect Jumps, and Loop Branches
 - Integrated Instruction Fetch Units



Branch History Table(BHT)

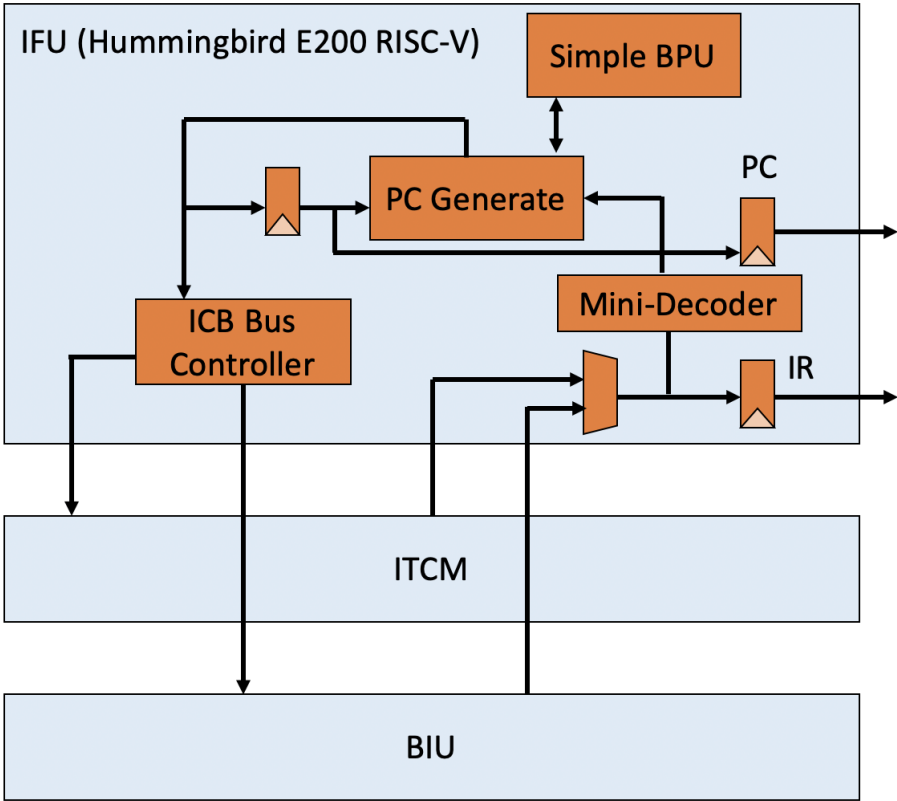
- ❑ 1-Bit Predictor
- ❑ 2-Bit Predictor

Branch-Target Buffers



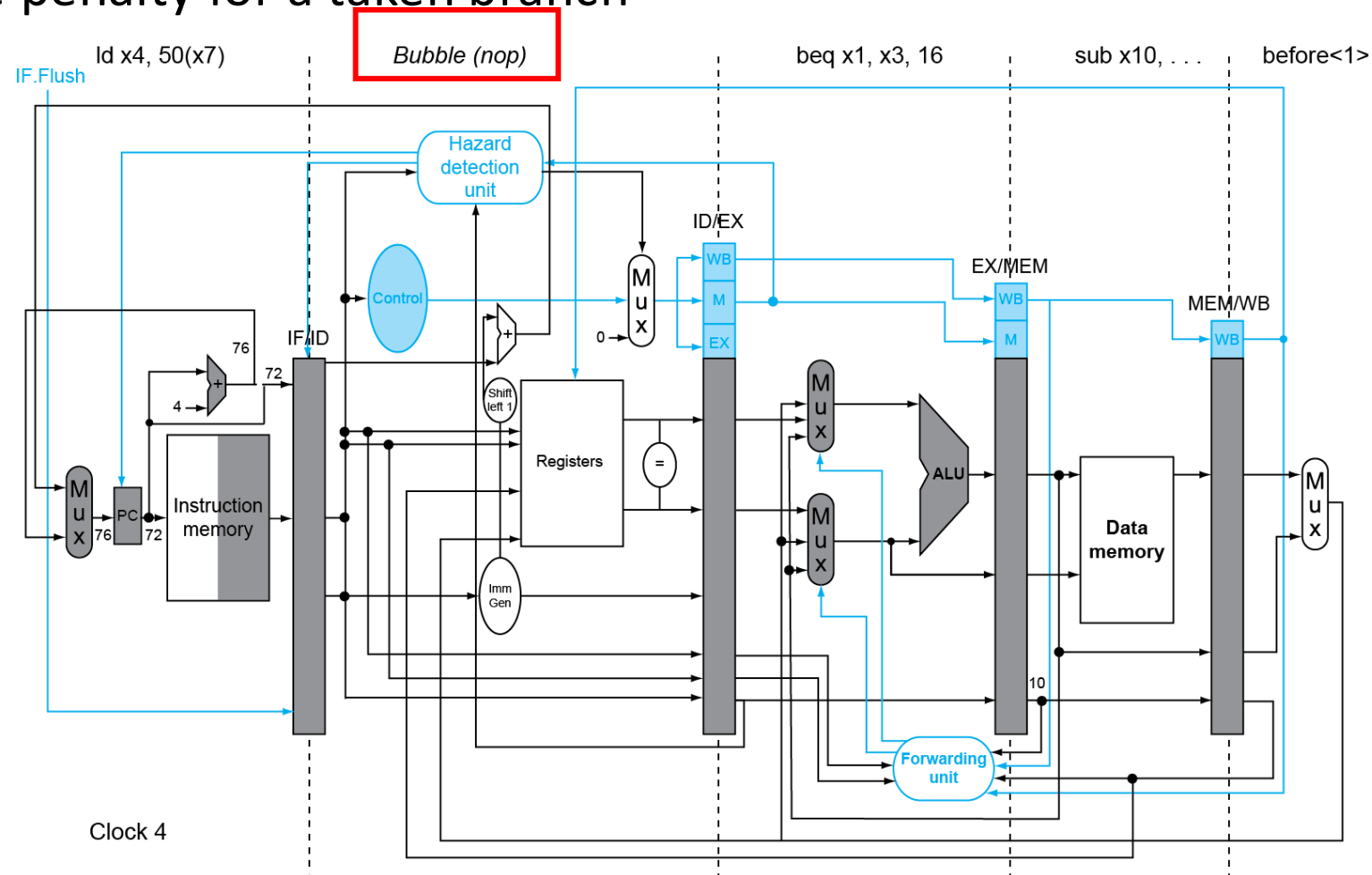
Integrated Instruction Fetch Units

- An integrated instruction fetch unit that integrates several functions:
 - Integrated branch prediction
 - Instruction prefetch
 - Instruction memory access and buffering
- Instruction fetch as a simple single pipe stage given the complexities of multiple issue is no longer valid.



Calculating the Branch Target

- Even with predictor, still need to calculate the target address
 - 1-cycle penalty for a taken branch



Calculating the Branch Target

- Even with predictor, still need to calculate the target address
 - 1-cycle penalty for a taken branch
- Branch target buffer
 - Cache of target addresses
 - Indexed by PC when instruction fetched
 - If hit and instruction is branch predicted taken, can fetch target immediately



Branch-Target Buffer/Branch-Target Cache

Is instruction in BTB?	Predict	Reality	Delay cycle
Yes	Taken	Taken	0
Yes	Taken	Not taken	2
No	Not taken	Taken	2
No	Not taken	Not taken	0

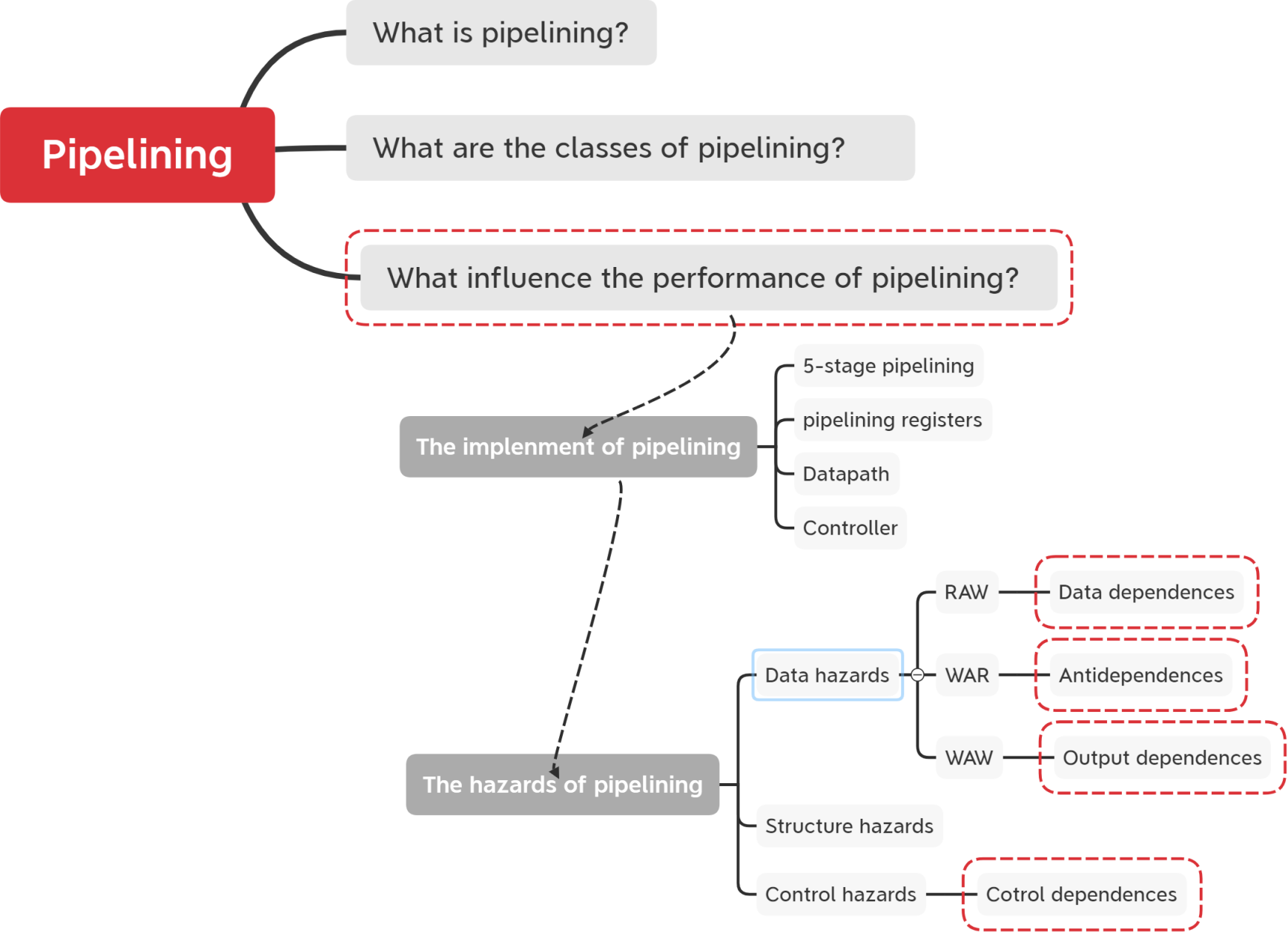


Branch-Target Buffer/Branch-Target Cache

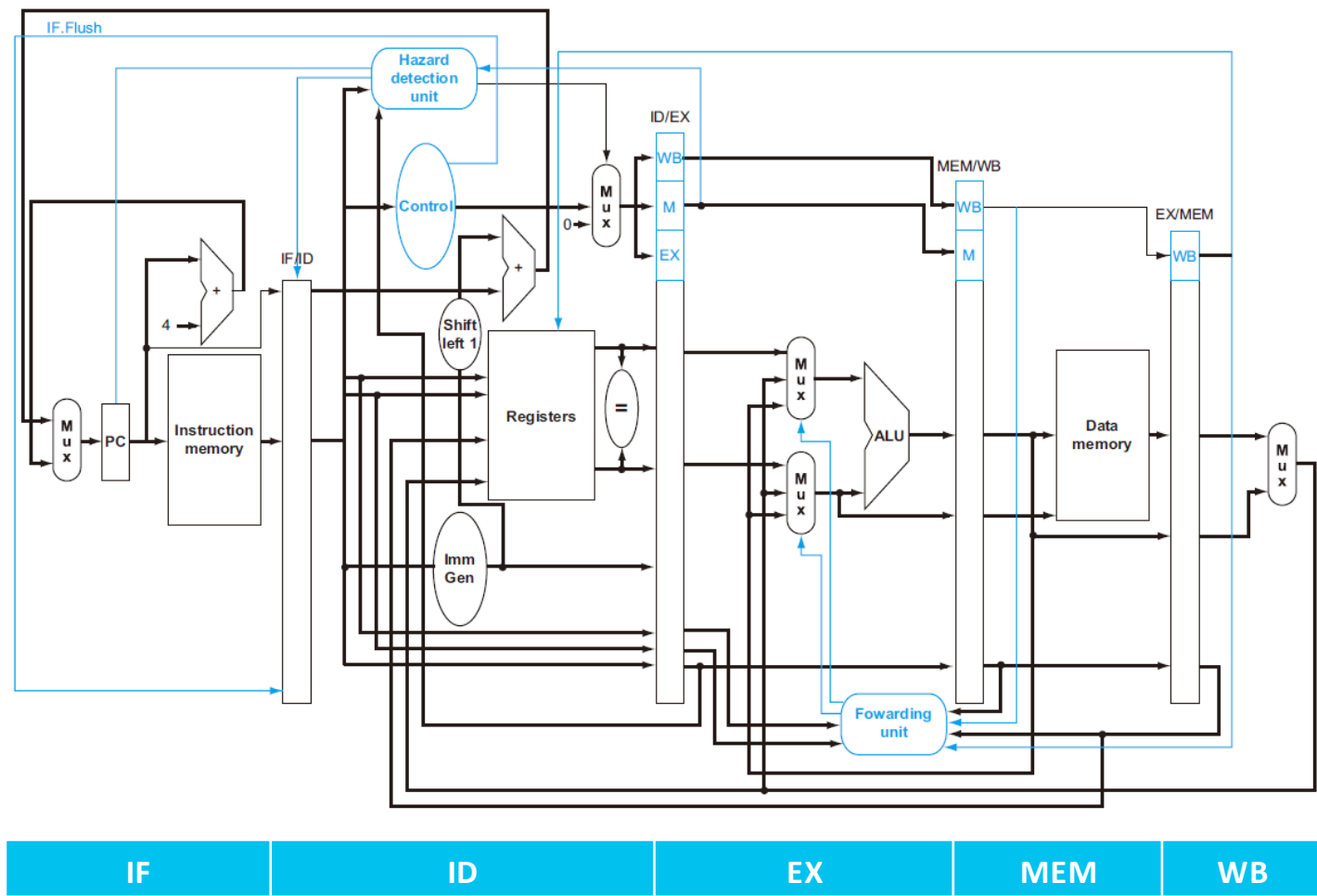
Benefit

- Get instructions at branch target faster
- It can provide multiple instructions at the branch target once, which is necessary for the multi processor;
- branch folding
 - It is possible to achieve unconditional branching without delay, or sometimes conditional branching without delay.



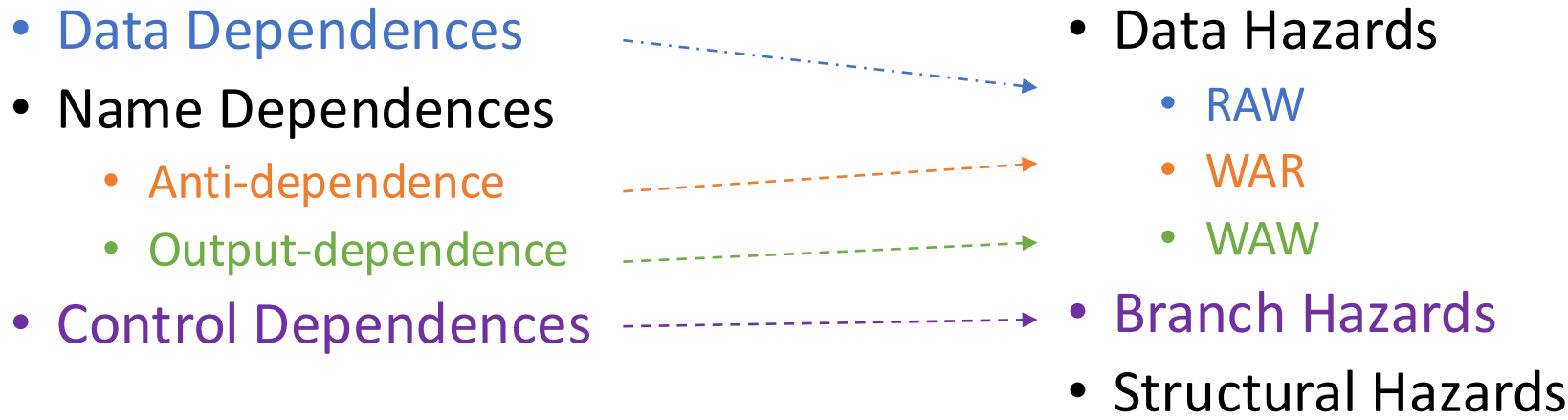


The Classic Five-Stage Pipeline for a RISC Processor



The Classic Five-Stage Pipeline for a RISC Processor

- A Simple Implementation of RISC-V Dependences are a property of *programs*.
- Instructions Dependences • Pipeline Hazards



Hazard are properties of the *pipeline organization*.



The Classic Five-Stage Pipeline for a RISC Processor

Consider this code:

```
FADD.D R1, R2, R4
FADD.D R2, R1, 1
FSUB.D R1, R4, R5
```

- (1) Point out all the pipeline Hazards (RAW, WAR, WAW).**
- (2) Analyze the hazards and give your solutions.**



Dynamic Scheduling

A major **limitation** of simple pipelining techniques is that:

- they use in-order instruction issue and execution
- For example, consider this code:

```
FDIV.D      F4, F0, F2
FSUB.D      F10, F4, F6
FADD.D      F12, F6, F14
```

The **FADD.D** instruction cannot execute because the **dependence of FSUB.D on FDIV.D** causes the pipeline to **stall**; yet, **FADD.D** is not data dependent on anything in the pipeline.

Instructions are issued in program order, and if an instruction is stalled in the pipeline no later instructions can proceed.



Dynamic Scheduling

- Out-of-order execution introduces the possibility of WAR and WAW hazards, which do not exist in the five-stage integer pipeline and its logical extension to an in-order floating-point pipeline.
 - Consider the following RISC-V floating-point code sequence:

WAW

[

FDIV.D F10, F0, F2

FSUB.D F10, F4, F6

FADD.D F6, F8, F14

]

WAR

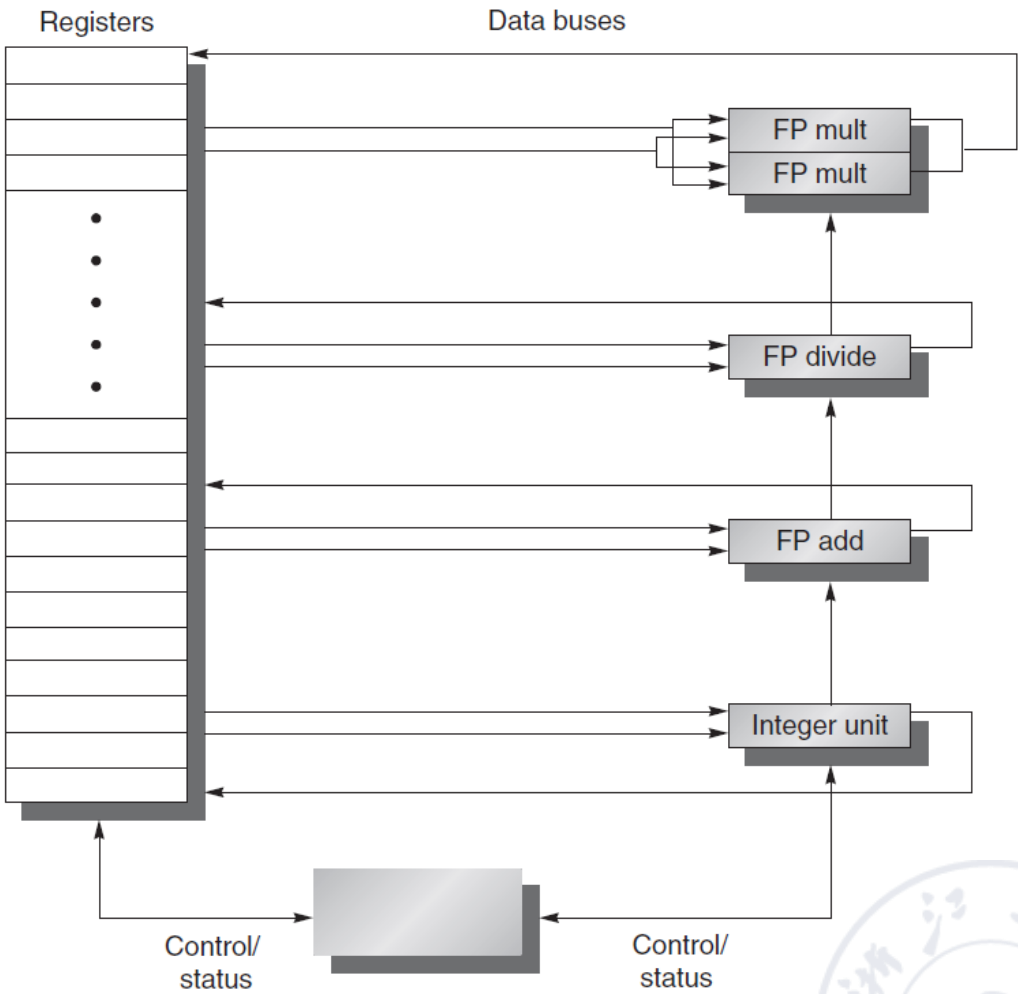
- Scoreboard algorithm is an approach to schedule the instructions.
- Robert Tomasulo introduces register renaming in hardware to minimize WAW and WAR hazards, named Tomasulo's Approach.



Dynamic Scheduling

Idea: Dynamic Scheduling

Method: **out-of-order** execution

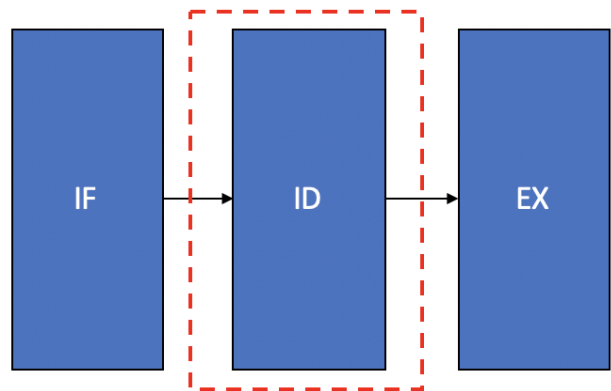


Dynamic Scheduling with a Scoreboard



Dynamic Scheduling

- A Simple Implementation of RISC-V



Check **structural** hazards

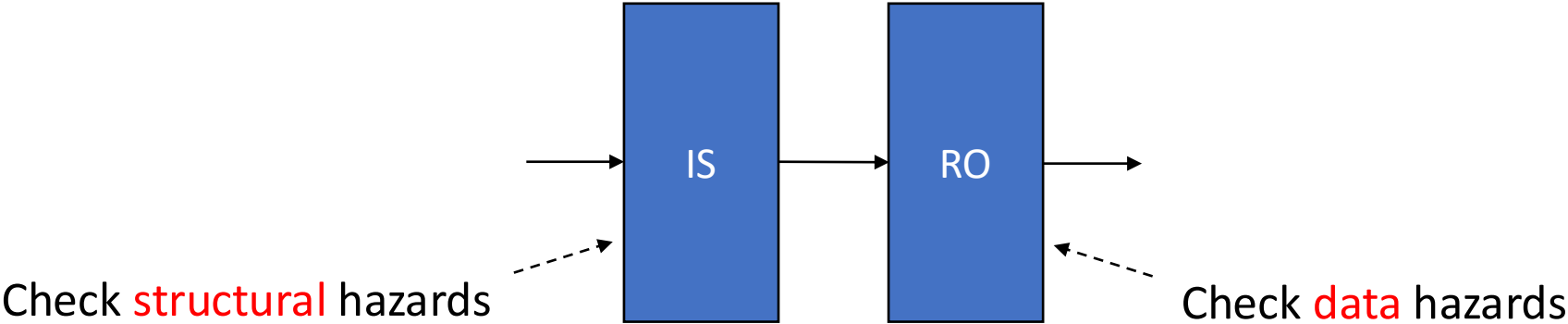
Check **data** hazards

- When an instruction could execute without hazards, it was issued from ID knowing that all data hazards had been resolved.

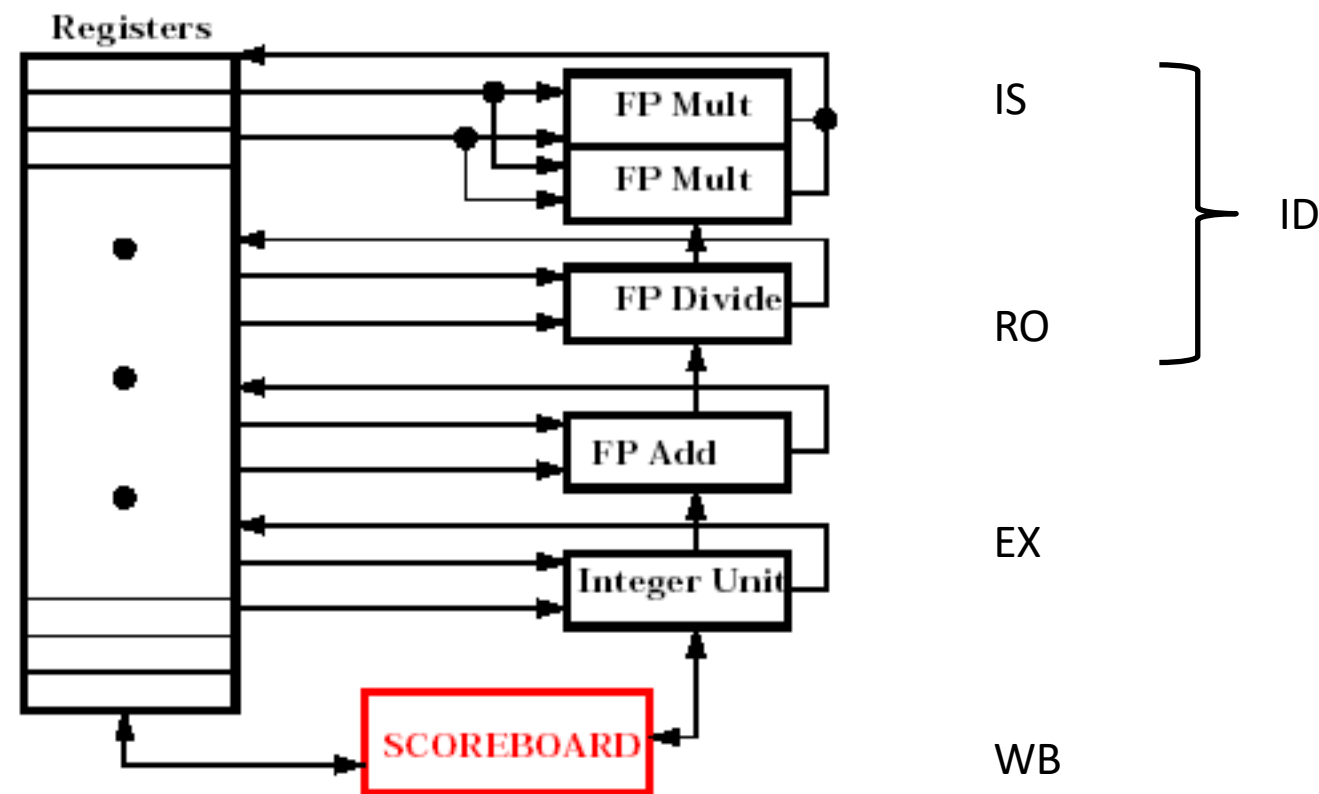


Dynamic Scheduling

- To allow out-of-order execution, we essentially split the ID pipe stage of our simple five-stage pipeline into two stages:
 - Issue(IS): Decode instructions, check for structural hazards. (in-order issue)
 - Read Operands(RO): Wait until no data hazards, then read operands. (out of order execution)



Dynamic Scheduling: Scoreboard algorithm



The basic structure of a processor with scoreboard



Dynamic Scheduling: Scoreboard algorithm

- Show what the information tables look like for the following code sequence when only the first load has completed and written its result:

FLD	F6, 34 (R2)
FLD	F2, 45 (R3)
FMUL.D	F0, F2, F4
FSUB.D	F8, F6, F2
FDIV.D	F10, F0, F6
FADD.D	F6, F8, F2



Dynamic Scheduling: Scoreboard algorithm

Instruction	Instruction Status			
	IS	RO	EX	WB
FLD F6, 34(R2)	✓	✓	✓	✓
FLD F2, 45(R3)	✓	✓	✓	
FMUL.D F0, F2, F4	✓			
FSUB.D F8, F6, F2	✓			
FDIV.D F10, F0, F6	✓			
FADD.D F6, F8, F2				



Name	Function Component Status								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	yes	Load	F2	R3				no	
Mult1	yes	MUL	F0	F2	F4	Integer		no	yes
Mult2	no								
Add	yes	SUB	F8	F6	F2		Integer	yes	no
Divide	yes	DIV	F10	F0	F6	Mult1		no	yes

Rj, Rk : “yes” ——operand is ready but not read;
 “no” & “Qj = null” ——operand is read;
 “no” & “Qj != null” ——operand is not ready.

	Register Status							
	F0	F2	F4	F6	F8	F10	...	F30
Qi	Mult1	Integer			Add	Divide		



Dynamic Scheduling: Scoreboard algorithm

Instruction	Instruction Status			
	IS	RO	EX	WB
FLD F6, 34(R2)	✓	✓	✓	✓
FLD F2, 45(R3)	✓	✓	✓	✓
FMUL.D F0, F2, F4	✓	✓	✓	
FSUB.D F8, F6, F2	✓	✓	✓	✓
FDIV.D F10, F0, F6	✓			
FADD.D F6, F8, F2	✓	✓	✓	

Show what the status tables look like when the FMUL.D is ready to write its result.



Name	Function Component Status								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	yes	MUL	F0	F2	F4			no	no
Mult2	no								
Add	yes	ADD	F6	F8	F2			no	no
Divide	yes	DIV	F10	F0	F6	Mult1		no	yes

	Register Status							
	F0	F2	F4	F6	F8	F10	...	F30
Qi	Mult1			Add		Divide		

Show what the status tables look like when the FMUL.D is ready to write its result.



Dynamic Scheduling: Scoreboard algorithm

Instruction	Instruction Status			
	IS	RO	EX	WB
FLD F6, 34(R2)	✓	✓	✓	✓
FLD F2, 45(R3)	✓	✓	✓	✓
FMUL.D F0, F2, F4	✓	✓	✓	✓
FSUB.D F8, F6, F2	✓	✓	✓	✓
FDIV.D F10, F0, F6	✓	✓	✓	
FADD.D F6, F8, F2	✓	✓	✓	✓

Show what the status tables look like when the FDIV.D is ready to write its result.



Name	Function Component Status								
	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	no								
Mult1	no								
Mult2	no								
Add	no								
Divide	yes	DIV	F10	F0	F6			no	no

	Register Status							
	F0	F2	F4	F6	F8	F10	...	F30
Qi						Divide		

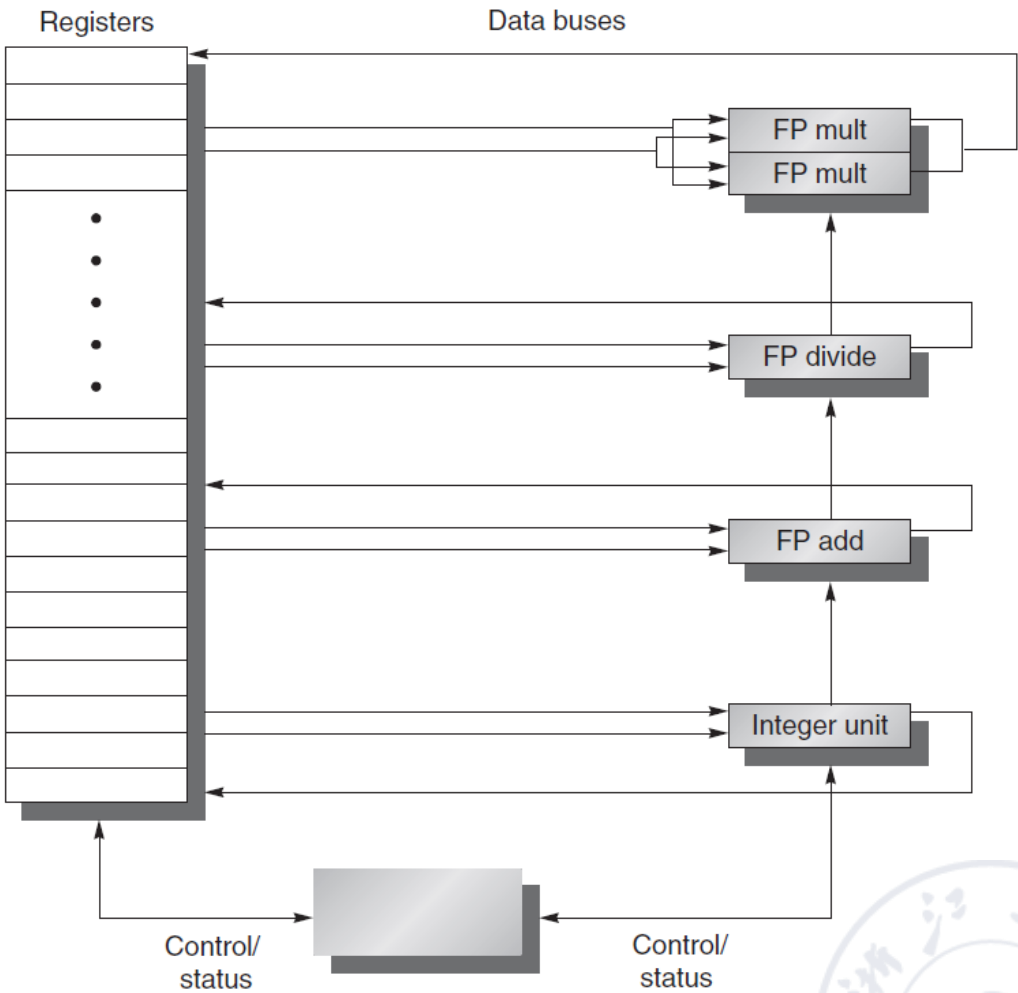
Show what the status tables look like when the FDIV.D is ready to write its result.



Dynamic Scheduling

Idea: Dynamic Scheduling

Method: **out-of-order** execution



Dynamic Scheduling with a Scoreboard



Practice in class

Suppose:

Add instruction needs 2 clock cycles. Multiply instruction needs 10 clock cycles.
Division instruction needs 40 clock cycles. LD instruction need 1 clock cycles.

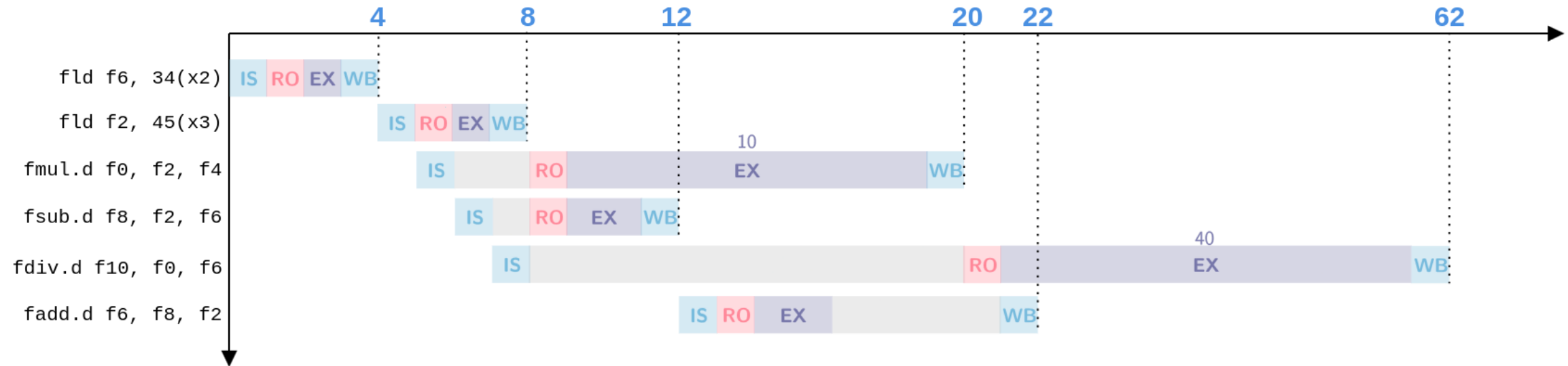
FLD	F6, 34 (R2)
FLD	F2, 45 (R3)
FMUL.D	F0, F2, F4
FSUB.D	F8, F6, F2
FDIV.D	F10, F0, F6
FADD.D	F6, F8, F2

How many clock cycles does it take to finish each instruction using scoreboard algorithm?

Scoreboard algorithm

	IS	RO	EX	WB
LD	1	1	1	1
MUL	1	1	10	1
SUB	1	1	2	1
DIV	1	1	40	1
ADD	1	1	2	1

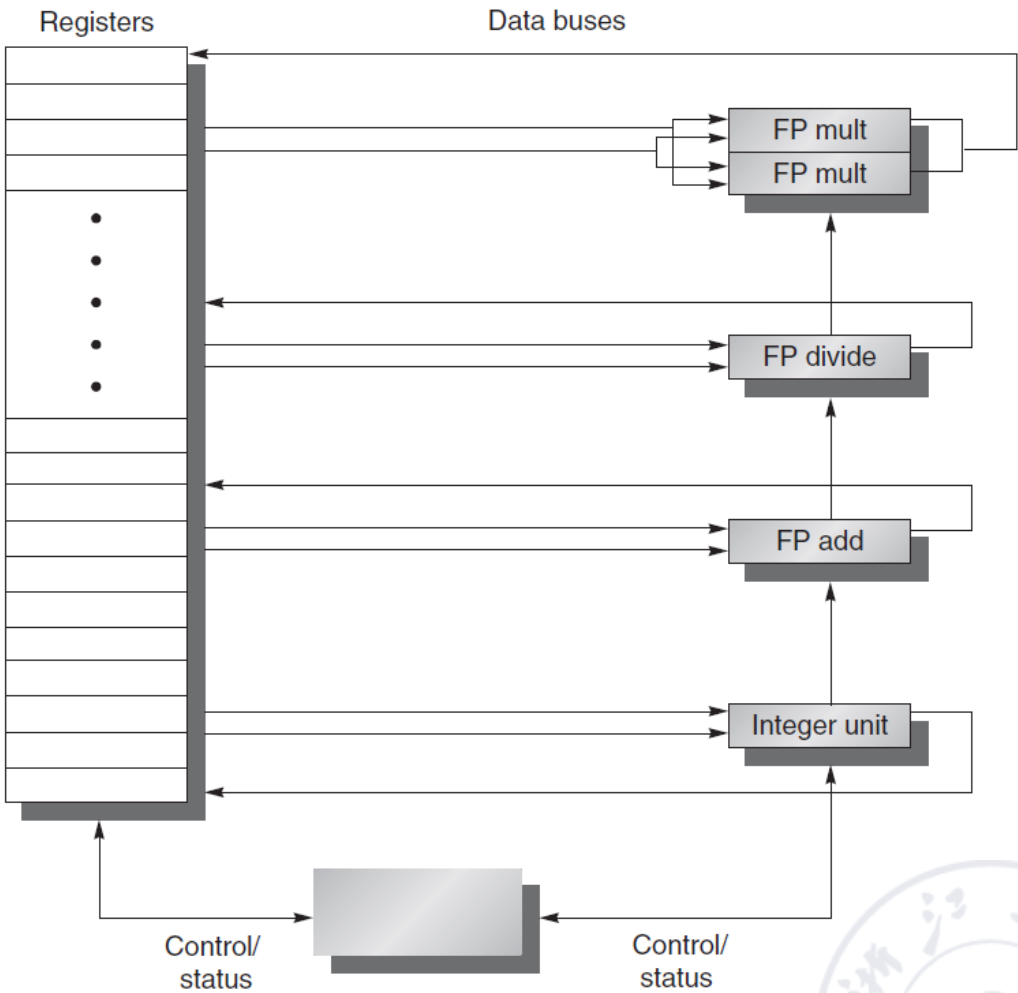
inst	Fi	Fj	Fk	is	ro	ex	wb
L.D	F6	34+R2		1	2	3	4
L.D	F2	45+R3		5	6	7	8
MUL.D	F0	F2	F4	6	9	10~19	20
SUB.D	F8	F2	F6	7	9	10~11	12
DIV.D	F10	F0	F6	8	21	22~61	62
ADD.D	F6	F8	F2	13	14	15~16	22



Dynamic Scheduling

Idea: Dynamic Scheduling

Method: **out-of-order** execution



Dynamic Scheduling with a Scoreboard



Dynamic Scheduling: The Idea

- Consider the following RISC-V floating-point code sequence:



Tomasulo's Approach

- These name dependences can all be eliminated by register renaming.
 - Assume the existence of two temporary registers, S and T.
 - The sequence can be rewritten without any dependences as :

FDIV.D F0, F2, F4

FADD.D S, F0, F8

FSD S, 0(R1)

FSUB.D T, F10, F14

FMUL.D F6, F10, T

}

F6 change as S

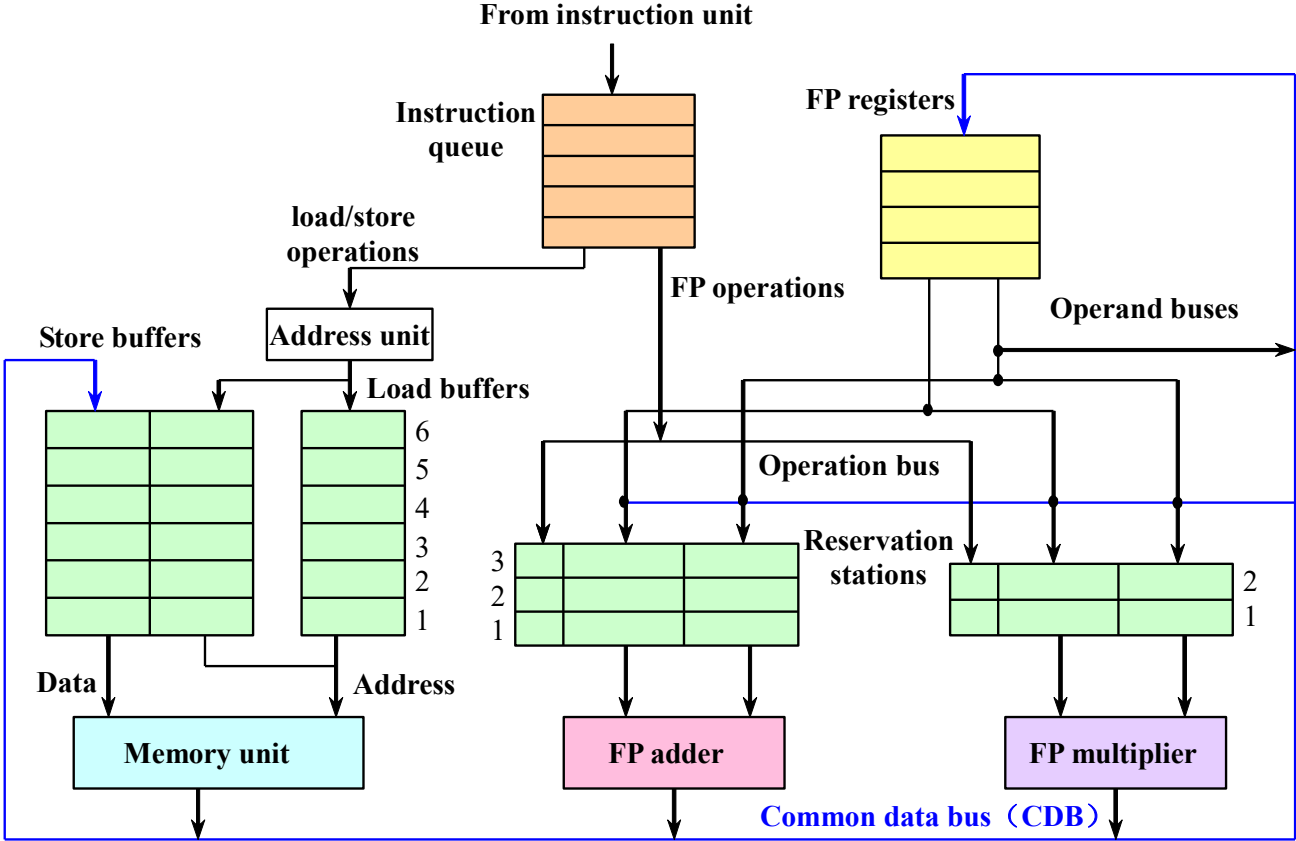
}

F8 change as T

Who finish the register renaming and how?



Tomasulo's Approach



The basic structure of a floating-point unit using Tomasulo's algorithm



Tomasulo's Approach: Main Idea

- It tracks when operands for instructions are available to minimize RAW hazards;
- It introduces register renaming in hardware to minimize WAW and WAR hazards.



Tomasulo's Approach

- Let's look at the **three steps** an instruction goes through:
 - Issue: Get the next instruction from the head of the instruction queue (FIFO)
 - If there is a matching **reservation station** that is **empty**, **issue** the instruction to the station with the operand values, if they are **currently in the registers**.
 - If there is **not an empty reservation station**, then there is a **structural hazard** and the instruction stalls until a station or buffer is freed.
 - If the operands are **not in the registers**, **keep track of the functional units** that will produce the operands.
- This step renames registers, eliminating WAR and WAW hazards not in the registers.



Tomasulo's Approach

Execute

- When all the operands are available, the operation can be executed at the corresponding functional unit.
- **Load** and **store** require a two-step execution process:
 - It computes the effective address when the base register is available.
 - The effective address is then placed in the **load** or **store** buffer.



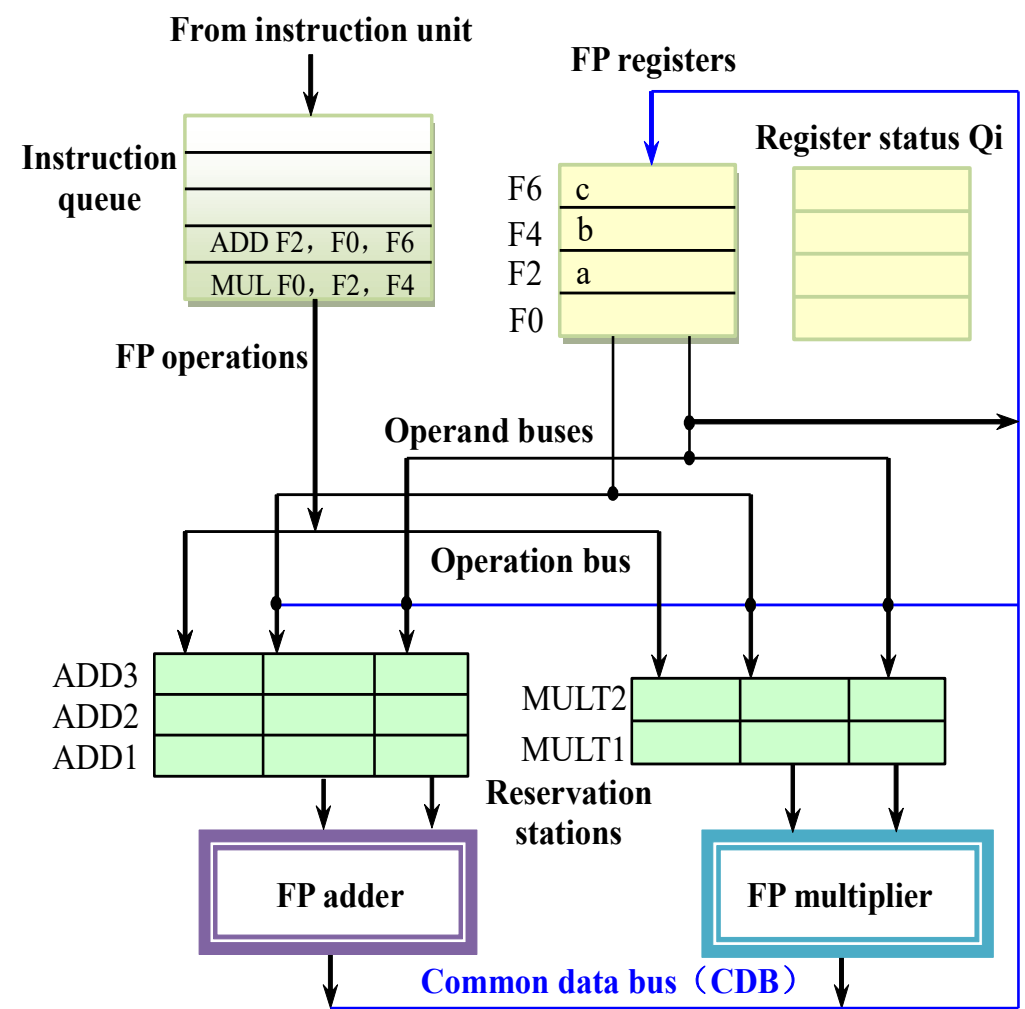
Tomasulo's Approach

Write results

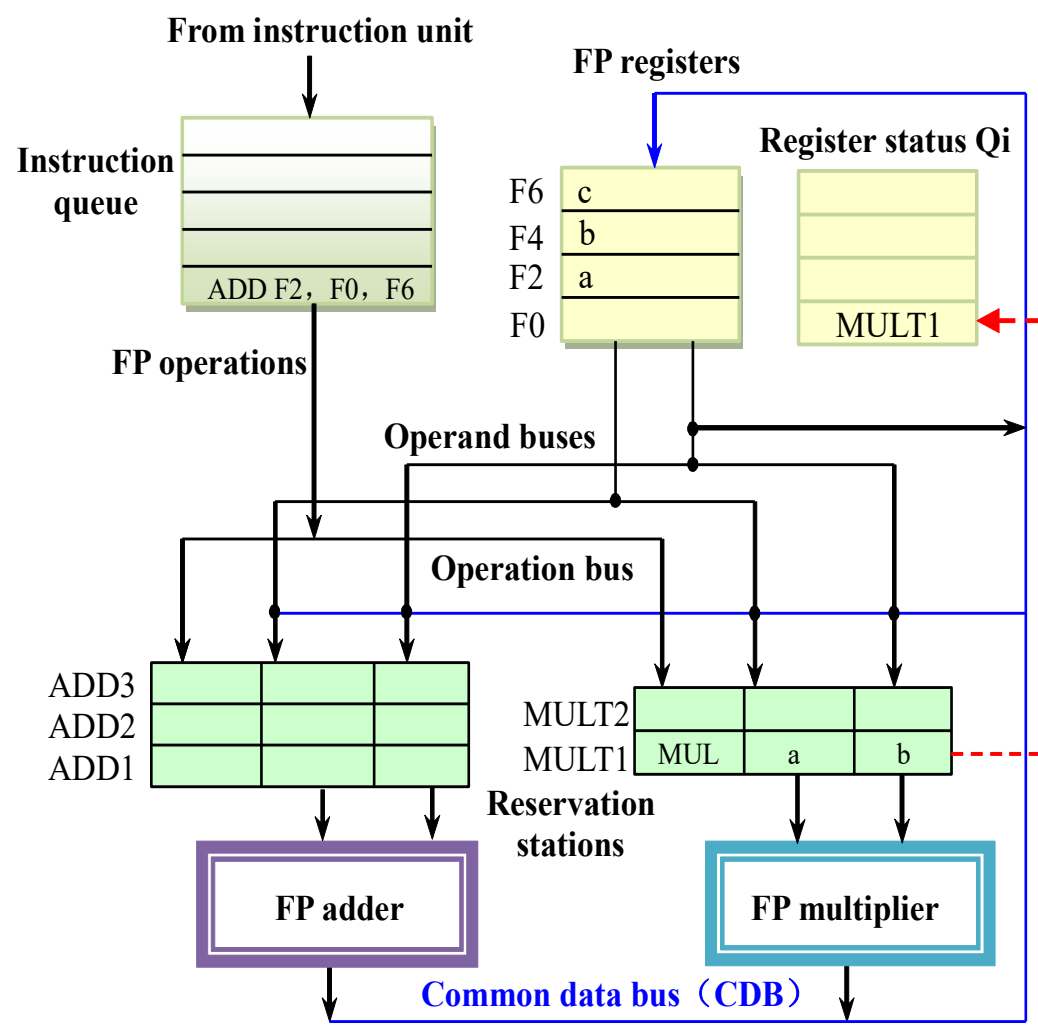
- When the result is available, write it on the **CDB** and from there into the **registers** and into any **reservation stations** (including **store** buffers).
- **Stores** are buffered in the store buffer until both the value to be stored and the store address are available, then the result is written as soon as the memory unit is free.



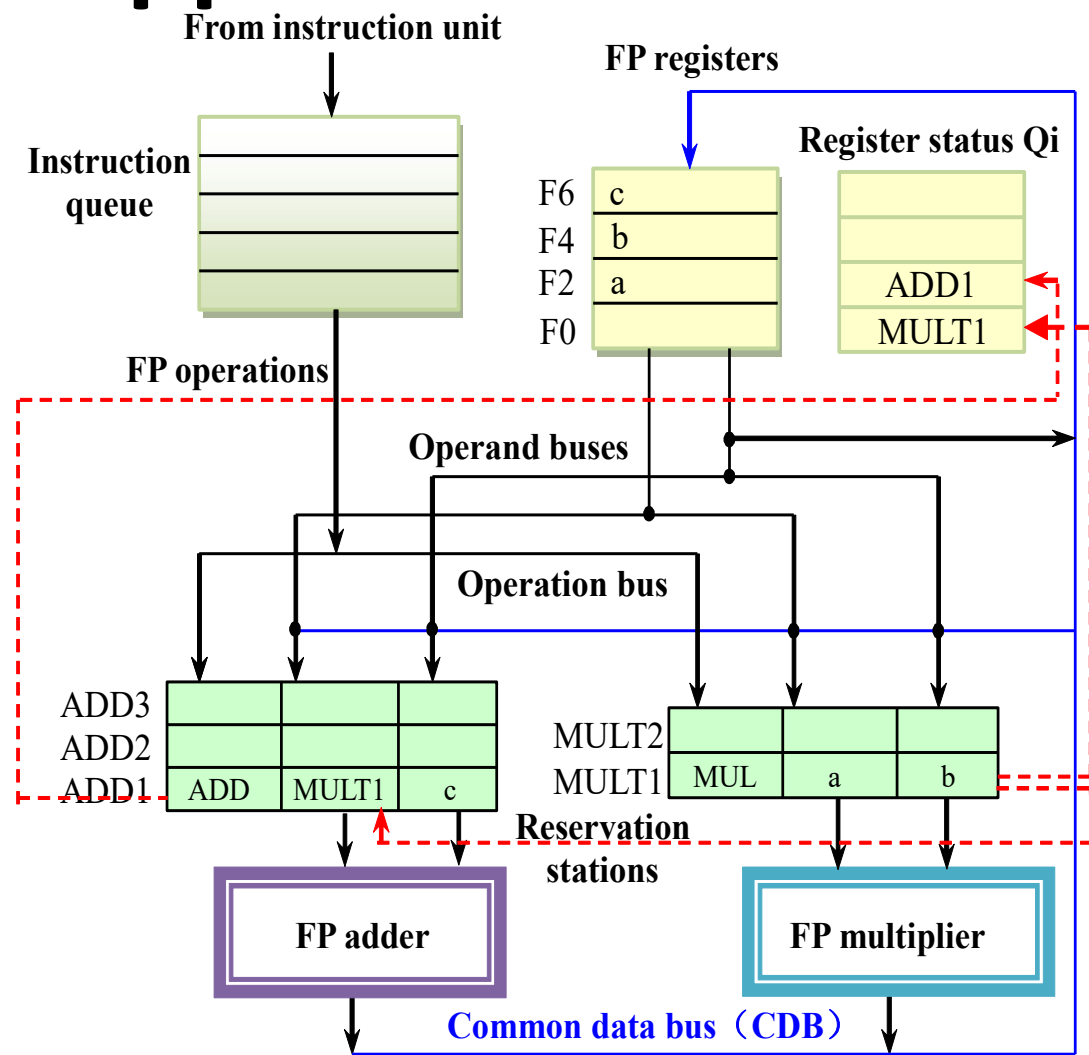
Tomasulo's Approach



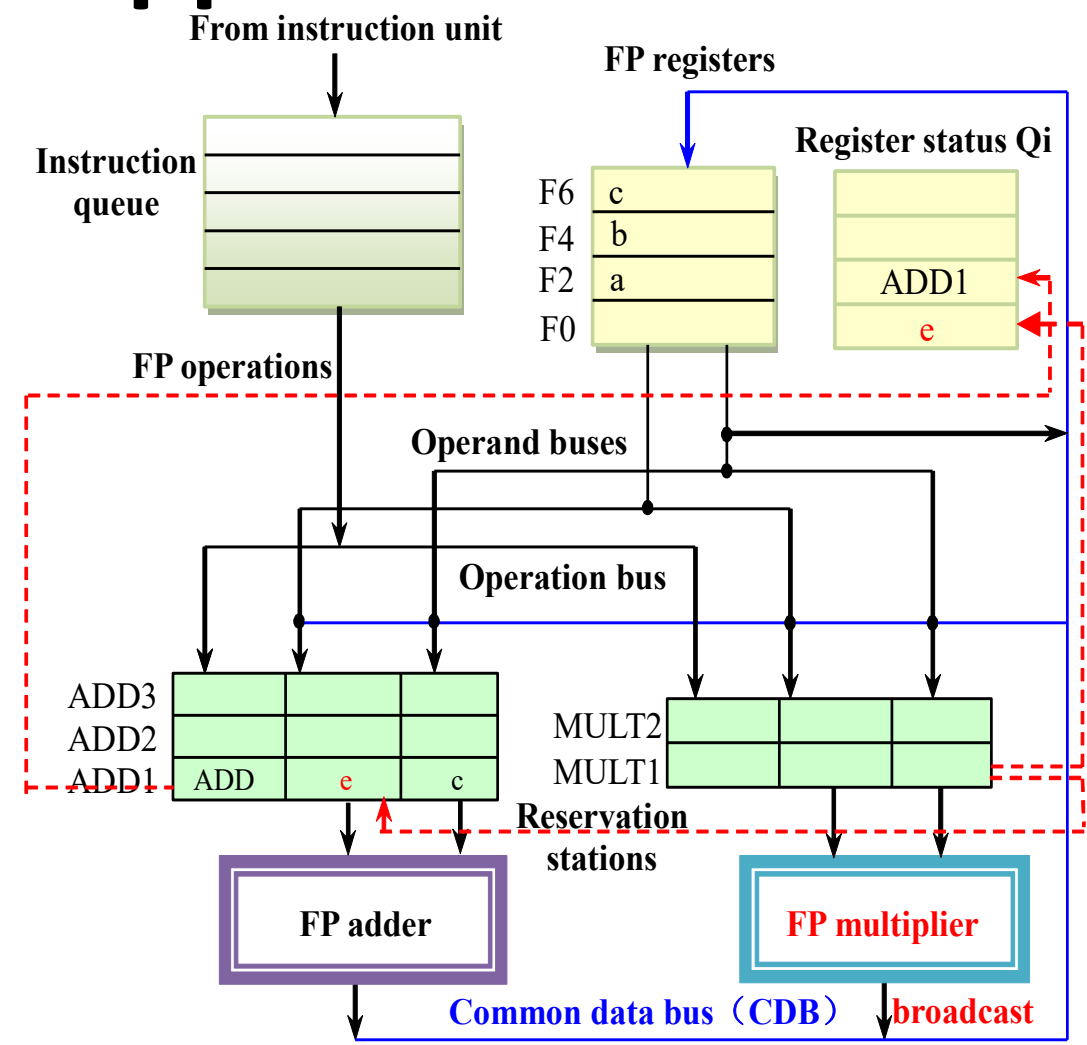
Tomasulo's Approach



Tomasulo's Approach



Tomasulo's Approach



Tomasulo's Approach

There are three tables for Tomasulo's Approach.

- **Instruction status table**: This table is included only to help you understand the algorithm; it is not actually a part of the hardware.
- **Reservation stations table**: The reservation station keeps the state of each operation that has issued.
- **Register status table (Field Qi)**: The number of the reservation station that contains the operation whose result should be stored into this register.



Tomasulo's Approach

Each reservation station has seven fields:

Op: The operation to perform on source operands.

Qj, Qk: The reservation stations that will produce the corresponding source operand.

Vj, Vk: The value of the source operands.

Busy: Indicates that this reservation station and its accompanying functional unit are occupied.

A: Used to hold information for the memory address calculation for a load or store.



Tomasulo's Algorithm and Examples

- Show what the information tables look like for the following code sequence when only the first load has completed and written its result:

FLD	F6, 34 (R2)
FLD	F2, 45 (R3)
FMUL.D	F0, F2, F4
FSUB.D	F8, F6, F2
FDIV.D	F10, F0, F6
FADD.D	F6, F8, F2



Dynamic Scheduling: Tomasulo's algorithm

Instruction		Instruction Status		
		Issue	Execute	Write Result
FLD	F6, 34(R2)	✓	✓	✓
FLD	F2, 45(R3)	✓	✓	
FMUL.D	F0, F2, F4	✓		
FSUB.D	F8, F6, F2	✓		
FDIV.D	F10, F0, F6	✓		
FADD.D	F6, F8, F2	✓		



Name	Function Component Status						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					45+Regs[R3]
Add1	Yes	SUB		Mem[34+Regs[R2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Reg[F4]	Load2		
Mult2	Yes	DIV		Mem[34+Regs[R2]]	Mult1		

	Register Status							
	F0	F2	F4	F6	F8	F10	...	F30
Qi	Mult1	Load2		Add2	Add1	Mult2		



Dynamic Scheduling: Tomasulo's algorithm

Instruction	Instruction Status		
	Issue	Execute	Write Result
FLD F6, 34(R2)	✓	✓	✓
FLD F2, 45(R3)	✓	✓	✓
FMUL.D F0, F2, F4	✓	✓	
FSUB.D F8, F6, F2	✓	✓	✓
FDIV.D F10, F0, F6	✓		
FADD.D F6, F8, F2	✓	✓	✓

Show what the status tables look like when the FMUL.D is ready to write its result.



Name	Function Component Status						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	No						
Add1	No						
Add2	No						
Add3	No						
Mult1	Yes	MUL	Mem[45+Regs[R3]]	Reg[F4]			
Mult2	Yes	DIV		Mem[34+Regs[R2]]	Mult1		

	Register Status							
	F0	F2	F4	F6	F8	F10	...	F30
Qi	Mult1					Mult2		



Summary

1. The main contributions of Tomasulo's Algorithm

- Dynamic scheduling
- Register renaming---eliminating WAW and WAR hazards
- Load/store disambiguation
- Better than Scoreboard Algorithm



Summary

2. Shortcomings of Tomasulo's Algorithm

- Structural complexity.
- Its performance is limited by Common Data Bus.
- A load and a store **can safely be done out of order**, provided they access **different addresses**. If a load and a store access the same address, then either:
 - The load is before the store in program order and interchanging them results in a WAR hazard, or
 - The store is before the load in program order and interchanging them results in a RAW hazard
 - Interchanging two stores to the same address results in a WAW hazard



Summary

3. The limitations on ILP approaches directly led to the movement to multicore.

Question

Does **out-of-order execution** mean **out-of-order completion**?



Practice in class

Suppose:

Add instruction needs 2 clock cycles. Multiply instruction needs 10 clock cycles.

Division instruction needs 40 clock cycles. LD instruction need 1 clock cycles.

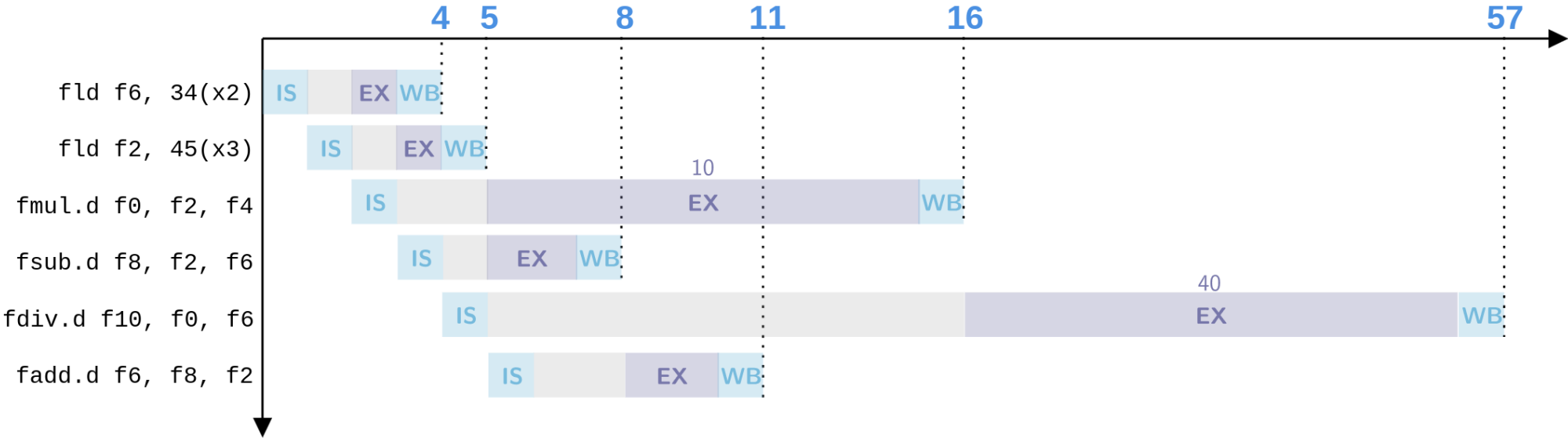
FLD	F6, 34 (R2)
FLD	F2, 45 (R3)
FMUL.D	F0, F2, F4
FSUB.D	F8, F6, F2
FDIV.D	F10, F0, F6
FADD.D	F6, F8, F2

How many cycles does it take to finish each instruction using Tomasulo's approach?

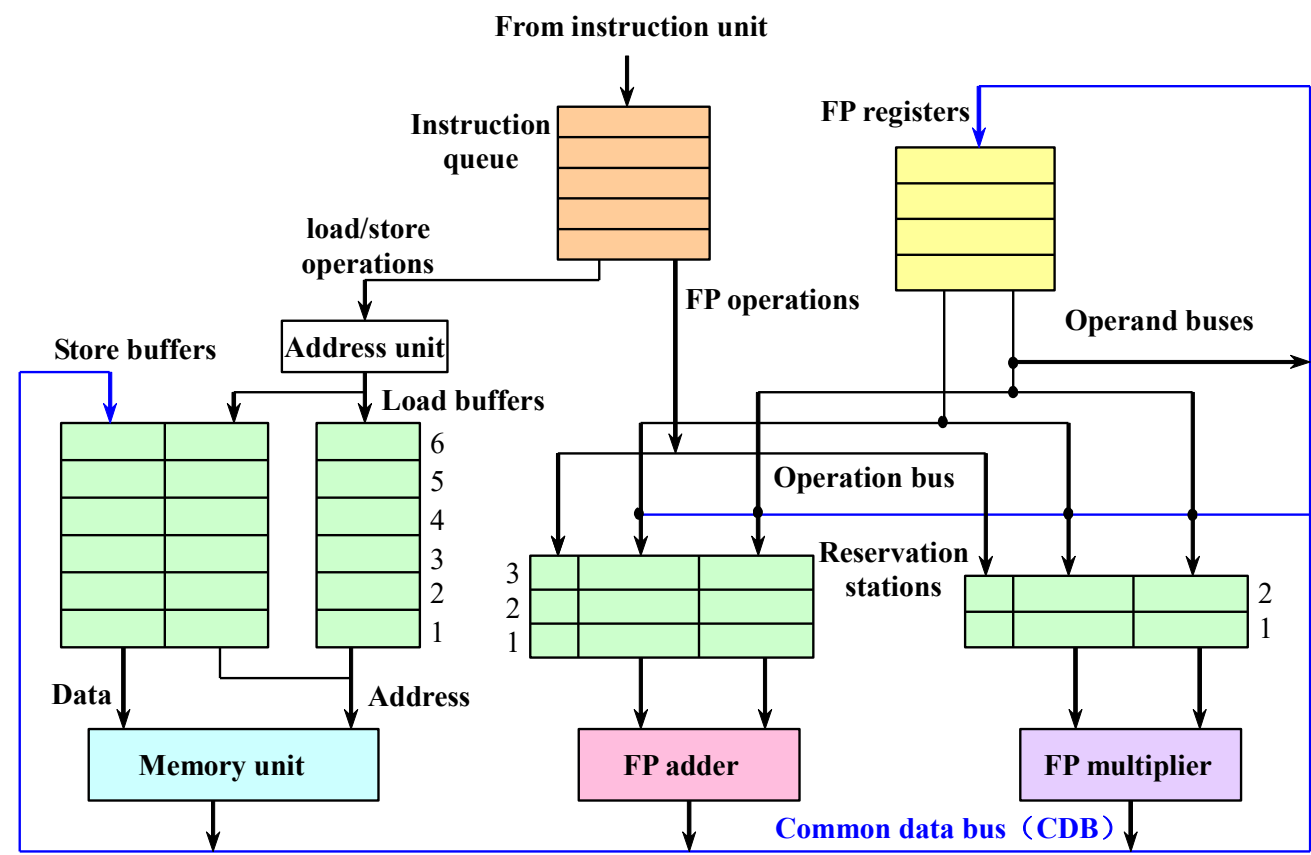
Tomasulo's approach

	IS	EX	WB
LD	1	1	1
MUL	1	10	1
SUB	1	2	1
DIV	1	40	1
ADD	1	2	1

inst	Fi	Fj	Fk	is	ex	wb
L.D	F6	34+R2		1	3	4
L.D	F2	45+R3		2	4	5
MUL.D	F0	F2	F4	3	6~15	16
SUB.D	F8	F2	F6	4	6~7	8
DIV.D	F10	F0	F6	5	17~56	57
ADD.D	F6	F8	F2	6	9~10	11



Tomasulo's Approach



The basic structure of a floating-point unit using Tomasulo's algorithm

